

BIT 4107 – Mobile Application Development

ENHANCED PROFESSIONAL STUDENT LOGBOOK

Student Name: ANDREW LEYIAPAS

Admission Number: BIT/2024/74067

School/Faculty: COMPUTER AND INFORMATICS

Programme: MOBILE APPLICATION

DEVELOPMENT

Academic Year: 2026

Semester: AUG/SEP

Lecturer: MICHAEL NYORO

Phone Number: 0111579951

Instructions

This logbook is used to document weekly learning activities, practical exercises, skills acquired, challenges encountered, solutions applied, and reflections. Students should complete all sections every week.

Each week to follow remarks in week 1 section.

Autogenerated Table of content

Week 1: Introduction to Mobile Application Development

Practical Activities Undertaken

- Installed Android Studio and configured development environment
- Created first Android project (STUDENT PORTALPRO)
- Learned basic structure of Android applications (Activities, XML layouts)
- Ran application on emulator for testing setup

Screenshots/Evidence Attached

only screenshots needed and labeling

Screenshots/Evidence Attached (Yes/No)

Android Studio installation/setup

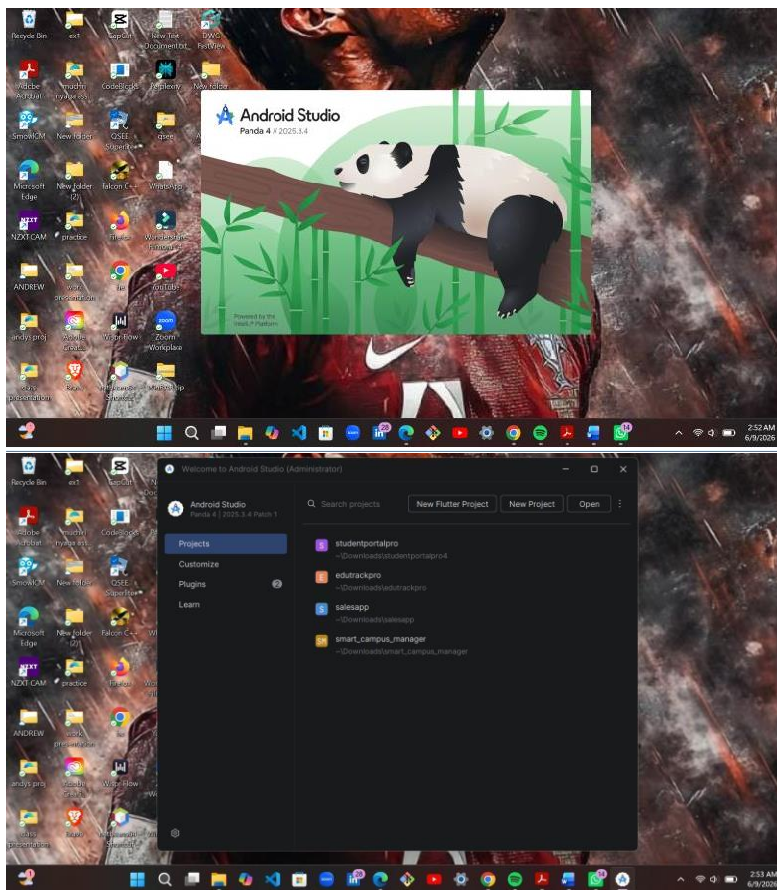


Figure 1. ANDROID STUDIO

INSTALLATIO

Created project structure

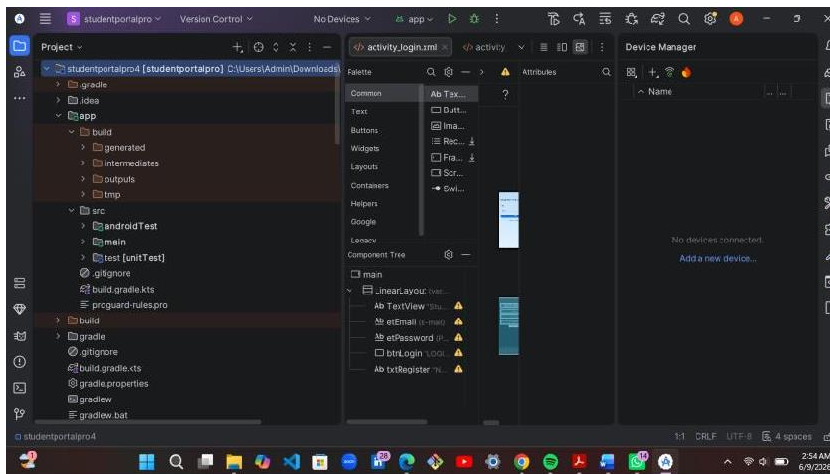


Figure 2.STUDENT PORTAL

PRO PROJECT CREATION

Emulator running

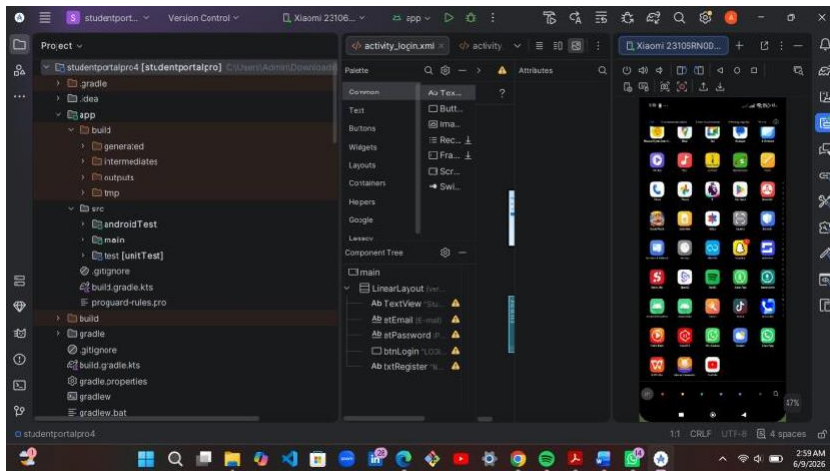


Figure 3.EMULATOR APP

RUNNING

Tools/Software Used

1. Android Studio
2. Java
3. Android Emulator
4. Windows OS

Personal Reflection

This week I learned how to set up Android Studio and create my first mobile application. I understood the basic project structure and how the emulator works for testing apps.

not more than 50 words

Week 2: Mobile Development Environment

Practical Activities Undertaken

- Designed login screen using XML
- Added Edit Text fields for username and password
- Created buttons for login and register
- Applied basic Constraint Layout for alignment

Screenshots/Evidence Attached

Login screen design

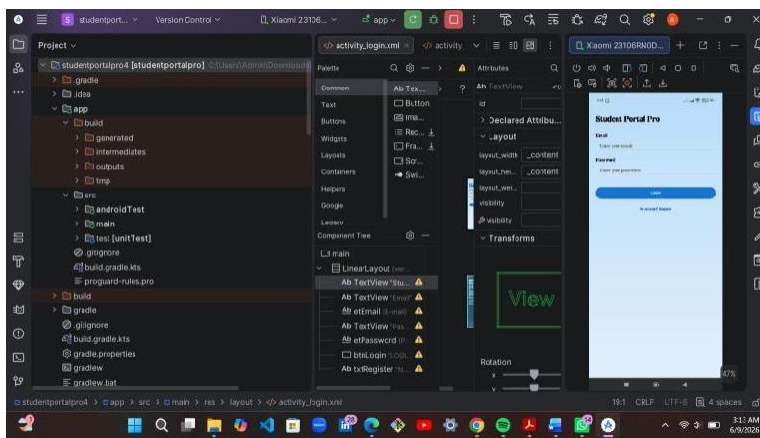
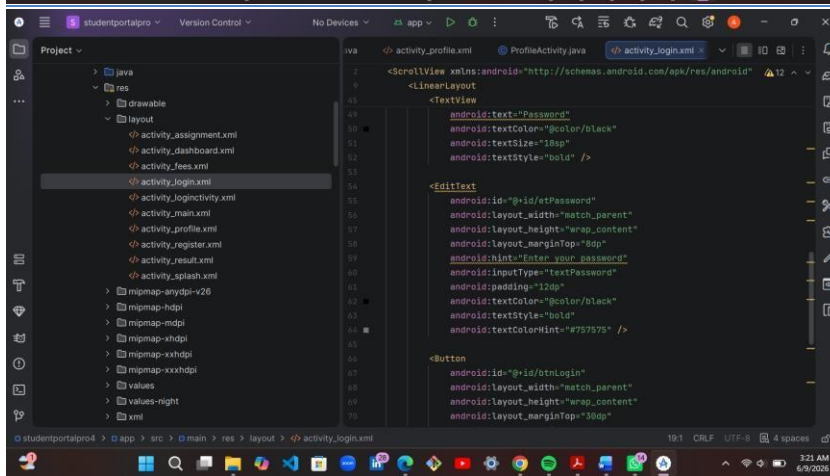
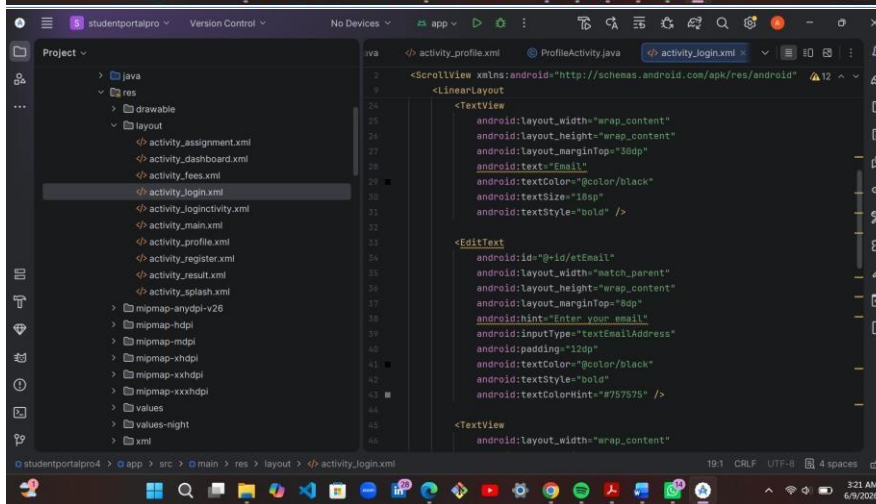
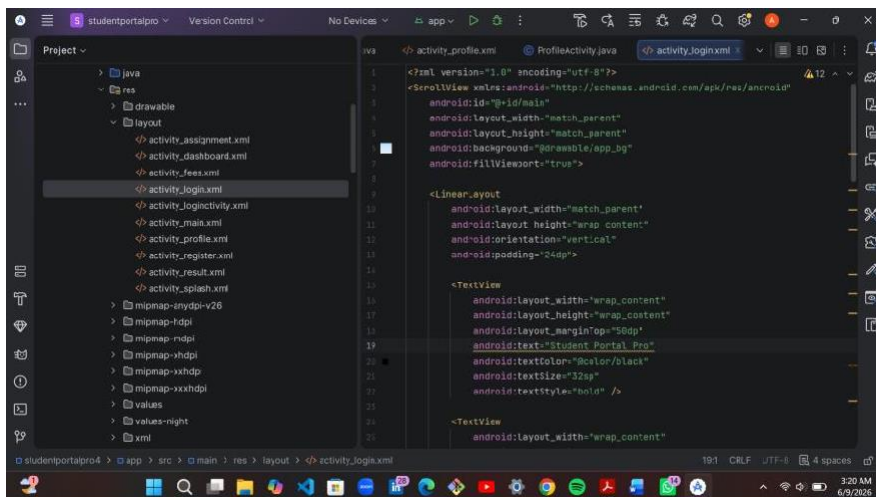


Figure 4.LOGIN SCREENS DESIGN

XML layout code



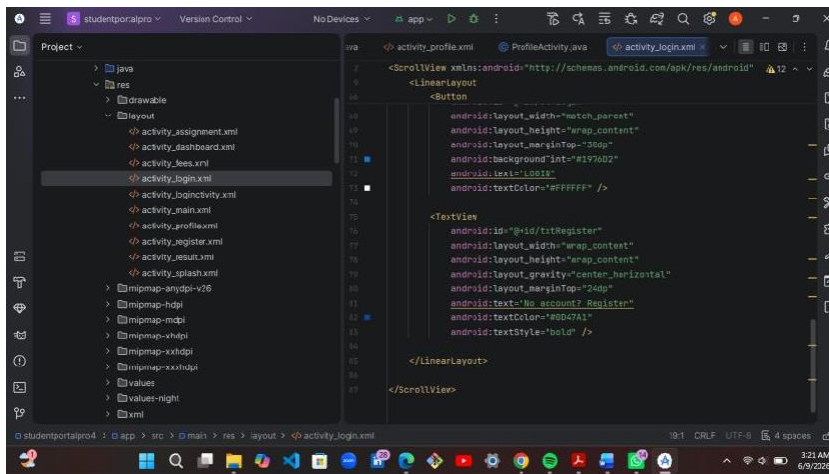


Figure 5.WHOLE

ACTIVITY_LOGIN.XML_CODE

Running UI on emulator

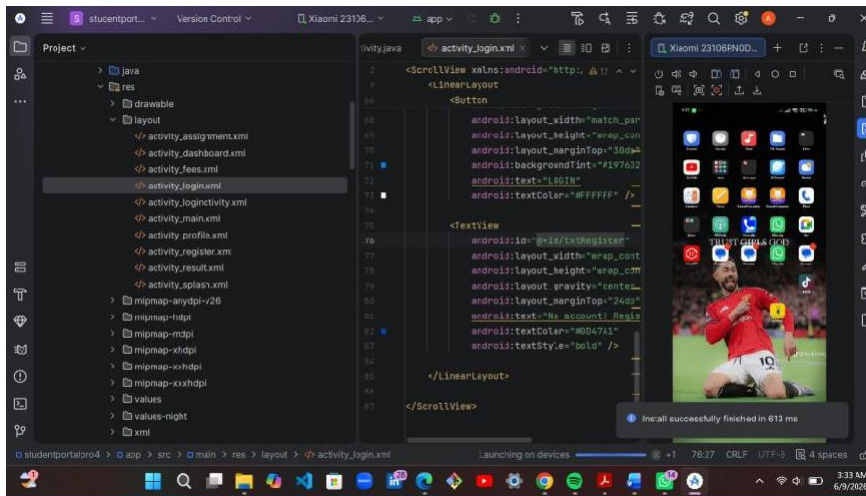


Figure 6.APP LAUNCHING

ON DEVICE

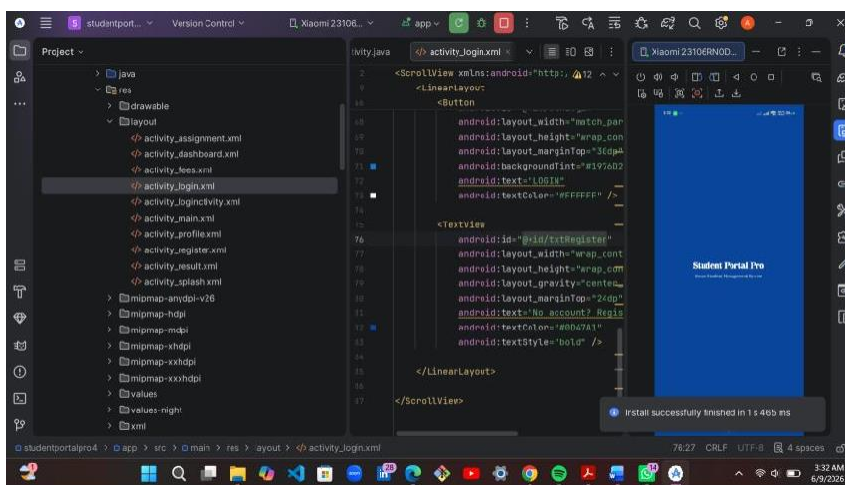


Figure 7.SUCCESSFULL

INSTALLATION

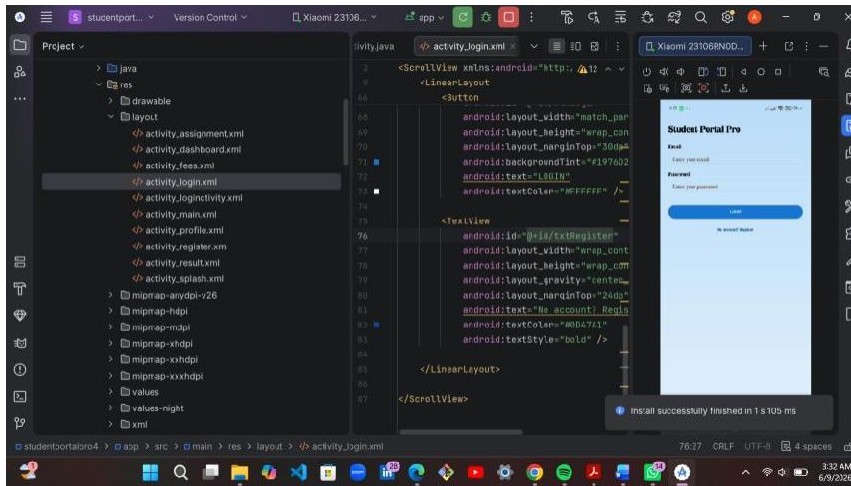


Figure 8.LOGIN SCREEN

ON EMULATOR VIEW

Tools/Software Used

- Android Studio
- XML
- Constraint Layout

Personal Reflection

This week I learned how to design user interfaces using XML in Android Studio. I also improved my understanding of layouts and positioning of UI components.

Week 3: User Interface Design

Practical Activities Undertaken

- Created dashboard screen for users
- Developed navigation between activities
- Added buttons for “Report Lost Item” and “Report Found Item”
- Improved UI design for better user experience

Dashboard screen

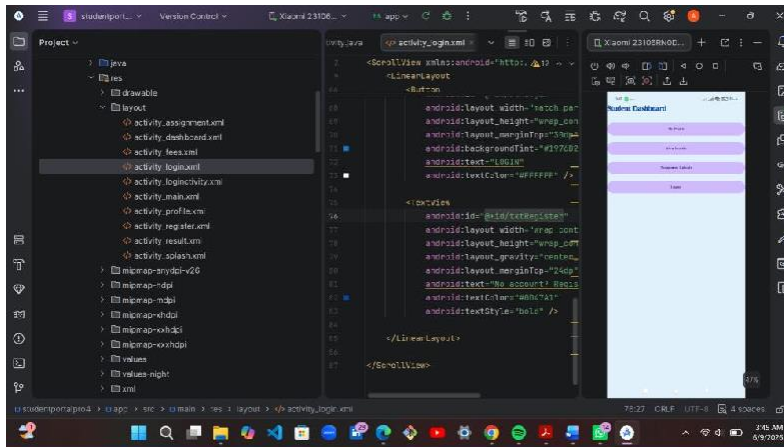


Figure 9. DASHBOARD SCREENS

Navigation between pages

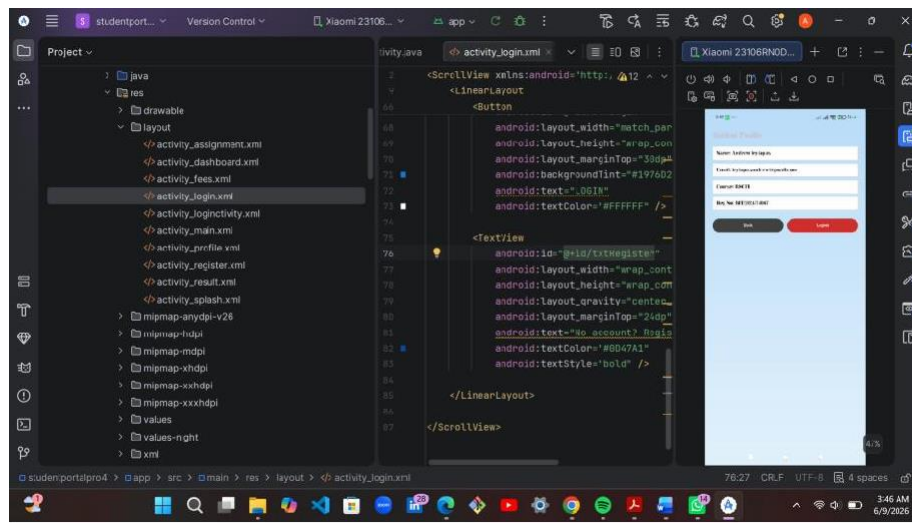


Figure 10.PROFILE BUTTON

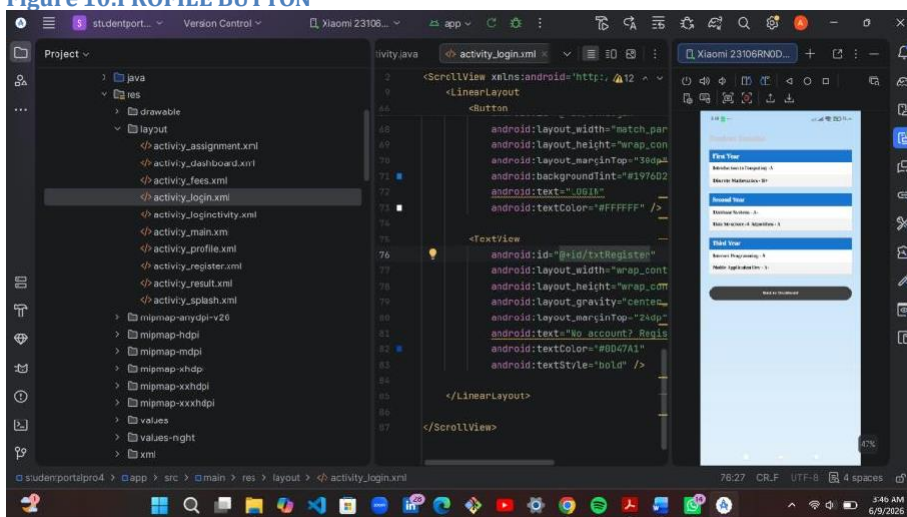


Figure 11.RESULTS BUTTON

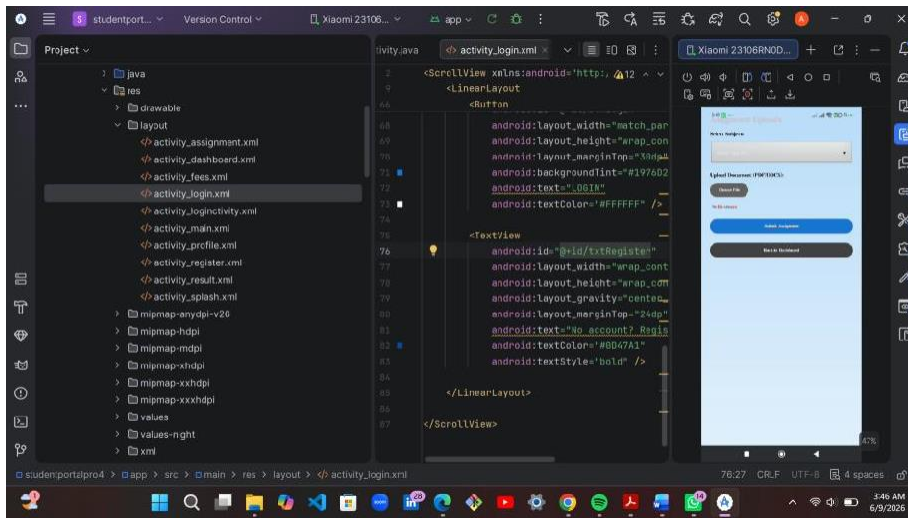
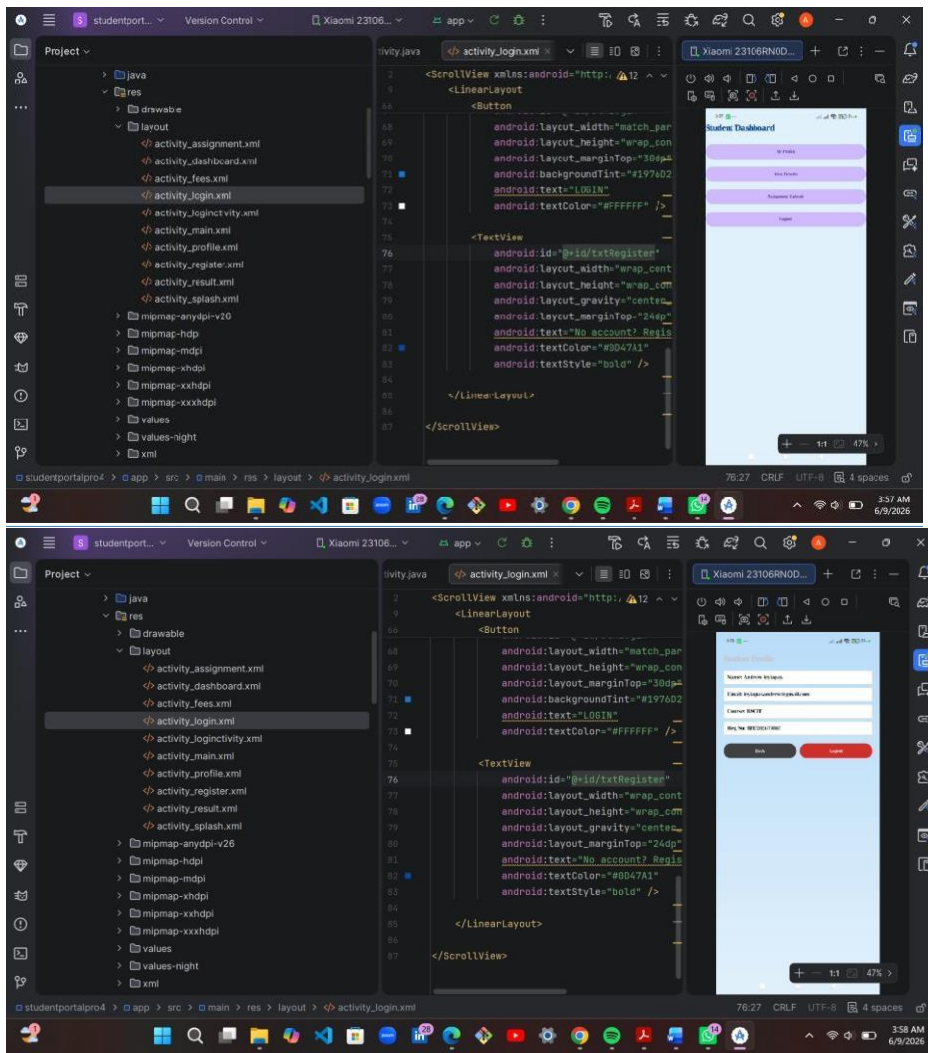


Figure 12.ASSIGNMENT BUTTON

Button interaction



Figure

13.DASHBOARD SCREENS &PROFILE BUTTONS&PROFILE SCREENS

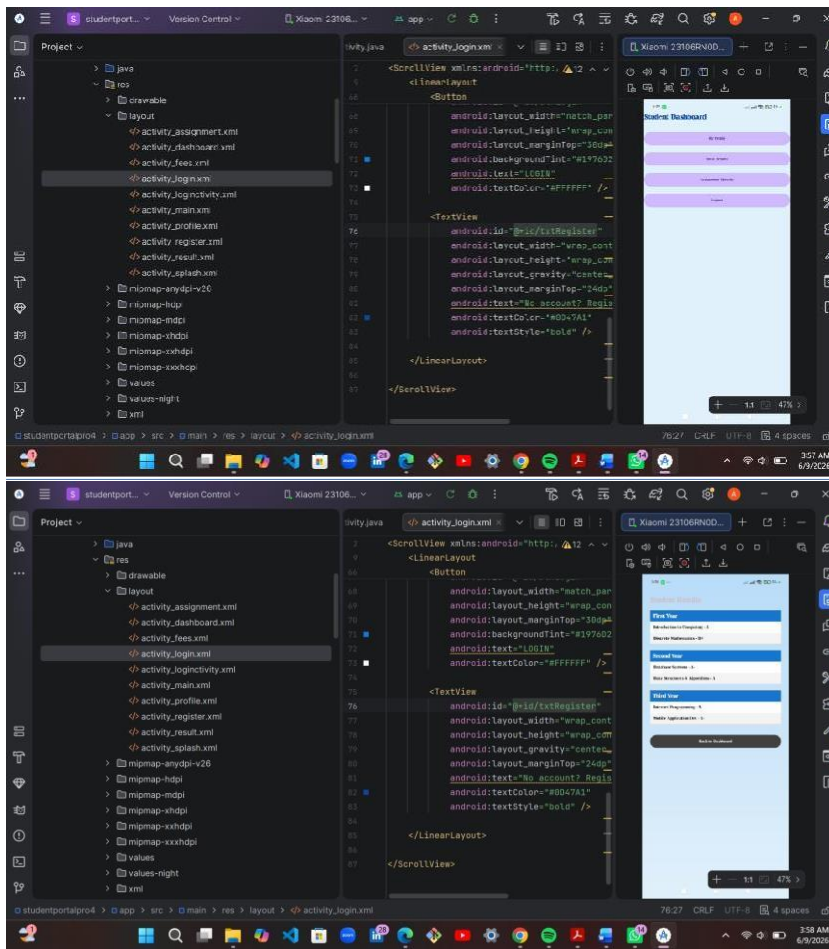
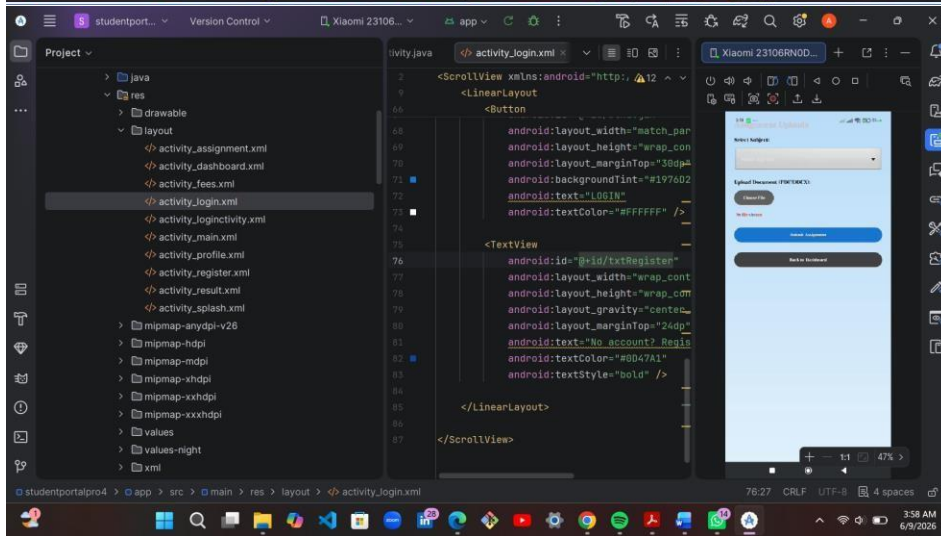
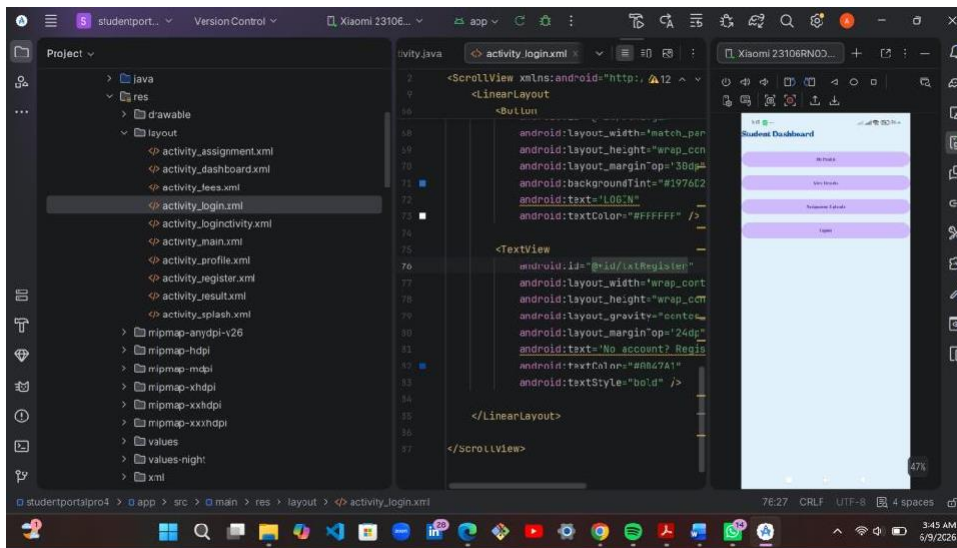


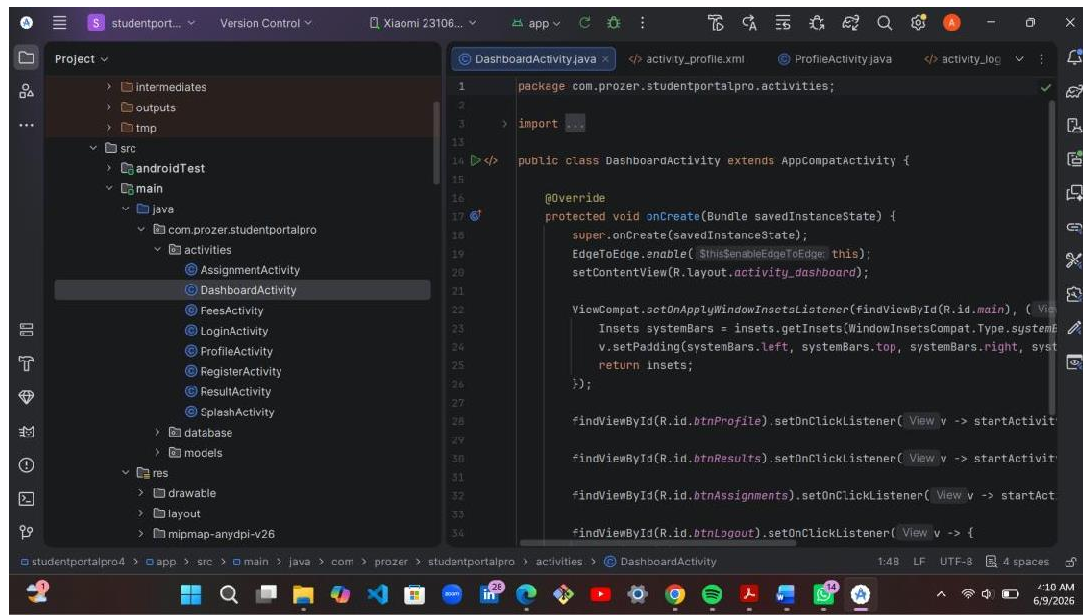
Figure 14.DASHBOARD

SCREENS&RESULTS BUTTON&RESULTS SCREENS

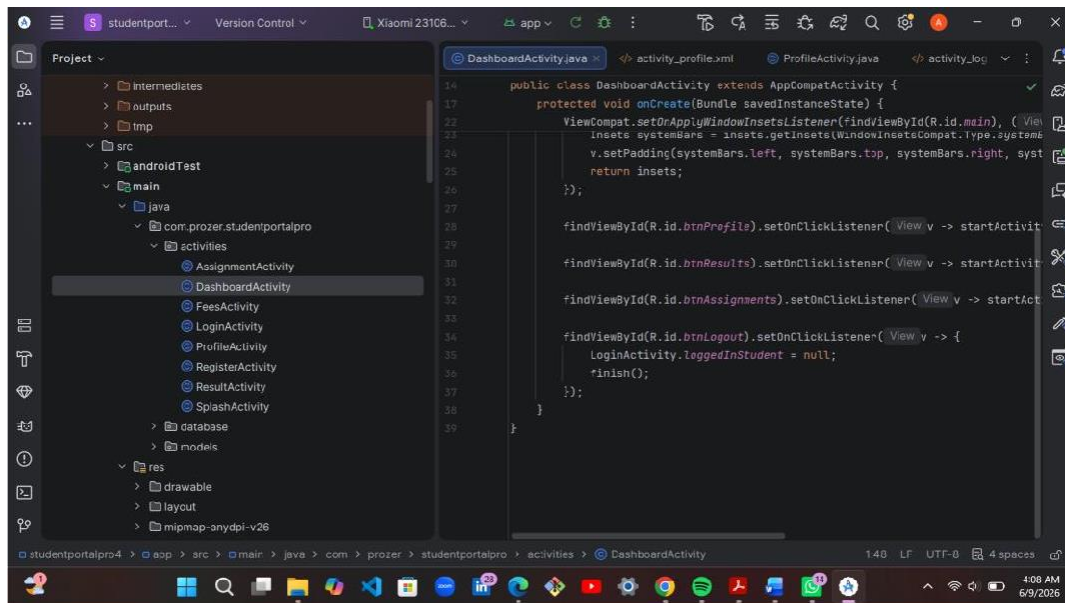


Figure

15.DASHBOARD SCREEN&ASSIGNMENT BUTTON&ASSIGNMENT SCREEN



```
1 package com.prozer.studentportalpro.activities;
2
3 import ...
4
5 public class DashboardActivity extends AppCompatActivity {
6
7     @Override
8     protected void onCreate(Bundle savedInstanceState) {
9         super.onCreate(savedInstanceState);
10        EdgeToEdge.enable(this);
11        setContentView(R.layout.activity_dashboard);
12
13        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (View v, Insets insets) -> {
14            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
15            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
16            return insets;
17        });
18
19        findViewById(R.id.btnProfile).setOnClickListener((View v) -> startActivity());
20        findViewById(R.id.btnResults).setOnClickListener((View v) -> startActivity());
21        findViewById(R.id.btnAssignments).setOnClickListener((View v) -> startActivity());
22        findViewById(R.id.btnLogout).setOnClickListener((View v) -> {
23            LoginActivity.loggedInStudent = null;
24            finish();
25        });
26    }
27 }
```



```
14 public class DashboardActivity extends AppCompatActivity {
15
16     @Override
17     protected void onCreate(Bundle savedInstanceState) {
18         super.onCreate(savedInstanceState);
19        EdgeToEdge.enable(this);
20        setContentView(R.layout.activity_dashboard);
21
22        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (View v, Insets insets) -> {
23            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
24            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
25            return insets;
26        });
27
28        findViewById(R.id.btnProfile).setOnClickListener((View v) -> startActivity());
29        findViewById(R.id.btnResults).setOnClickListener((View v) -> startActivity());
30        findViewById(R.id.btnAssignments).setOnClickListener((View v) -> startActivity());
31        findViewById(R.id.btnLogout).setOnClickListener((View v) -> {
32            LoginActivity.loggedInStudent = null;
33            finish();
34        });
35    }
36 }
```

Figure

16.JAVA CODE IMPLEMENTING BUTTON CLICK

Personal Reflection

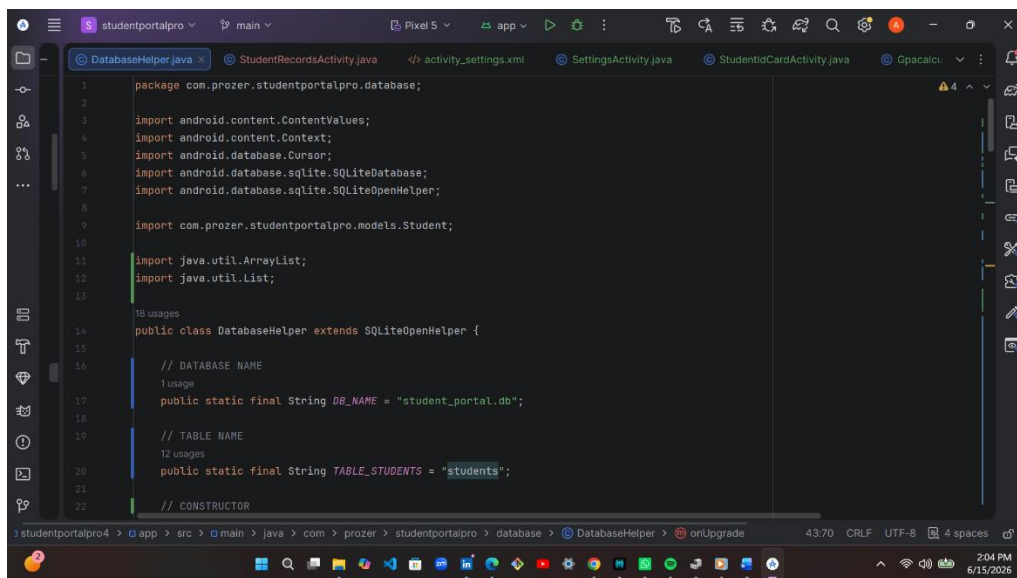
This week I learned how to connect multiple screens in an Android application. I also improved my understanding of navigation between activities and user interaction through buttons.

Week 4: Event Handling and User Interaction

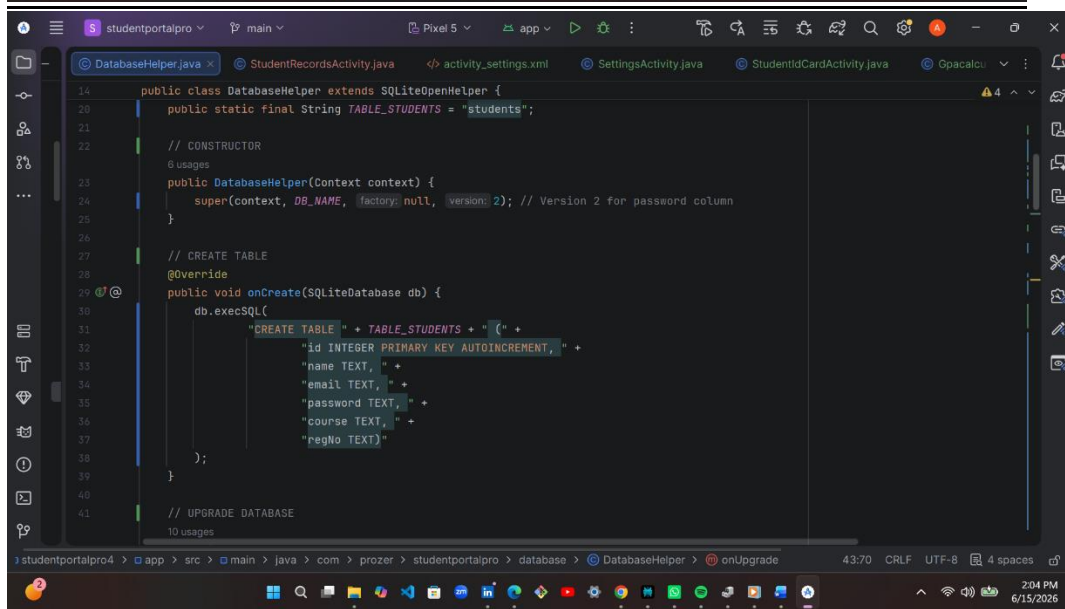
Practical Activities Undertaken

- Implemented SQLite database for student record management.
- Added Create, Read, Update and Delete (CRUD) operations.
- Developed RecyclerView for displaying student records.
- Implemented student search functionality.
- Added SharedPreferences for remembering login sessions.
- Improved dashboard with additional student services.

DatabaseHelper CRUD Methods



```
1 package com.prozer.studentportalpro.database;
2
3 import android.content.ContentValues;
4 import android.content.Context;
5 import android.database.Cursor;
6 import android.database.sqlite.SQLiteDatabase;
7 import android.database.sqlite.SQLiteOpenHelper;
8
9 import com.prozer.studentportalpro.models.Student;
10
11 import java.util.ArrayList;
12 import java.util.List;
13
14 public class DatabaseHelper extends SQLiteOpenHelper {
15
16     // DATABASE NAME
17     public static final String DB_NAME = "student_portal.db";
18
19     // TABLE NAME
20     public static final String TABLE_STUDENTS = "students";
21
22     // CONSTRUCTOR
```



```
23 // CONSTRUCTOR
24 public DatabaseHelper(Context context) {
25     super(context, DB_NAME, null, version: 2); // Version 2 for password column
26 }
27
28 // CREATE TABLE
29 @Override
30 public void onCreate(SQLiteDatabase db) {
31     db.execSQL(
32         "CREATE TABLE " + TABLE_STUDENTS + " (" +
33         "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
34         "name TEXT, " +
35         "email TEXT, " +
36         "password TEXT, " +
37         "course TEXT, " +
38         "regNo TEXT)");
39 }
40
41 // UPGRADE DATABASE
```

```
studentportalpro main Pixel 5 app
DatabaseHelper.java StudentRecordsActivity.java activity_settings.xml SettingsActivity.java StudentCardActivity.java Gpalcu
14 public class DatabaseHelper extends SQLiteOpenHelper {
40
41 // UPGRADE DATABASE
10 usages
42 @Override
43 @
44 public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
45 db.execSQL("DROP TABLE IF EXISTS " + TABLE_STUDENTS);
46 onCreate(db);
47
48 //
49 // AUTHENTICATION
50 //
51
52 //usage
53 public boolean registerStudent(String name, String email, String password, String course, String regNo) {
54 SQLiteDatabase db = this.getWritableDatabase();
55 ContentValues values = new ContentValues();
56 values.put("name", name);
57 values.put("email", email);
58 values.put("password", password);
59 values.put("course", course);
60 values.put("regNo", regNo);
61
62 long result = db.insert(TABLE_STUDENTS, nullColumnHack, null, values);
63
studentportalpro4 app src main java com prozer studentportalpro database DatabaseHelper onUpgrade 43/70 CRLF UTF-8 4 spaces
2:04 PM 6/15/2026
```

```
studentportalpro main Pixel 5 app
DatabaseHelper.java StudentRecordsActivity.java activity_settings.xml SettingsActivity.java StudentCardActivity.java Gpalcu
14 public class DatabaseHelper extends SQLiteOpenHelper {
40
41 // UPGRADE DATABASE
10 usages
42 @Override
43 @
44 public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
45 db.execSQL("DROP TABLE IF EXISTS " + TABLE_STUDENTS);
46 onCreate(db);
47
48 //
49 // AUTHENTICATION
50 //
51
52 //usage
53 public boolean registerStudent(String name, String email, String password, String course, String regNo) {
54 SQLiteDatabase db = this.getWritableDatabase();
55 ContentValues values = new ContentValues();
56 values.put("name", name);
57 values.put("email", email);
58 values.put("password", password);
59 values.put("course", course);
60 values.put("regNo", regNo);
61
62 long result = db.insert(TABLE_STUDENTS, nullColumnHack, null, values);
63
studentportalpro4 app src main java com prozer studentportalpro database DatabaseHelper onUpgrade 43/70 CRLF UTF-8 4 spaces
2:04 PM 6/15/2026
```

```
studentportalpro main Pixel 5 app
DatabaseHelper.java StudentRecordsActivity.java activity_settings.xml SettingsActivity.java StudentCardActivity.java Gpalcu
14 public class DatabaseHelper extends SQLiteOpenHelper {
52 public boolean registerStudent(String name, String email, String password, String course, String regNo) {
63
64
65 //usage
66 public Student loginStudent(String email, String password) {
67 SQLiteDatabase db = this.getReadableDatabase();
68 Cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_STUDENTS + " WHERE email=? AND password=?", new String[]{email, password});
69
70 if (cursor != null && cursor.moveToFirst()) {
71 Student student = new Student(
72 cursor.getInt(cursor.getColumnIndex(0)),
73 cursor.getString(cursor.getColumnIndex(1)),
74 cursor.getString(cursor.getColumnIndex(2)),
75 cursor.getString(cursor.getColumnIndex(4)), // course
76 cursor.getString(cursor.getColumnIndex(5)) // regNo
77 );
78 cursor.close();
79 return student;
80 }
81 return null;
82
83 //usage
84 public boolean updatePassword(int id, String newPassword) {
85
studentportalpro4 app src main java com prozer studentportalpro database DatabaseHelper onUpgrade 43/70 CRLF UTF-8 4 spaces
2:04 PM 6/15/2026
```

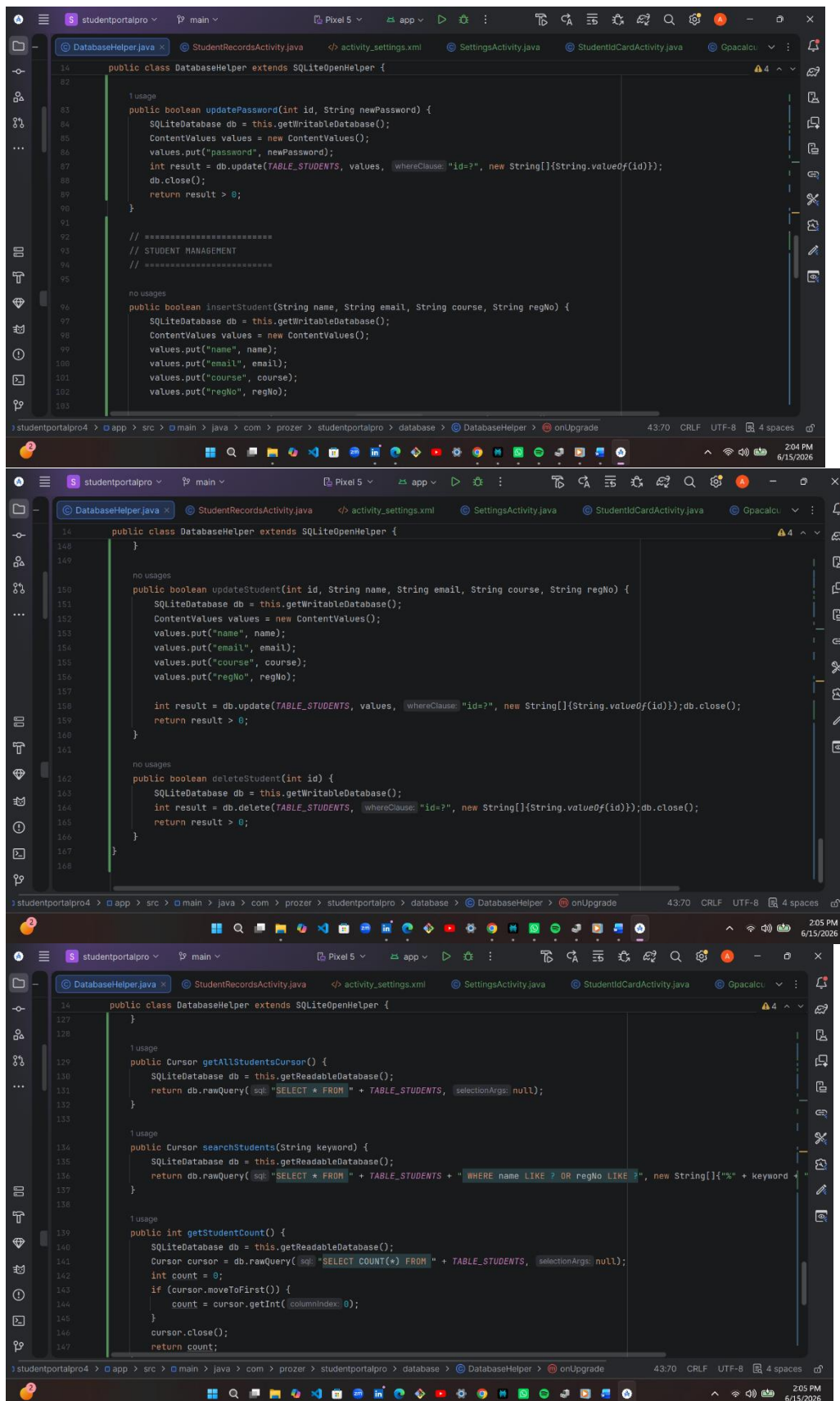


Figure 1SQL LITE CRUD operations

Registration Screen

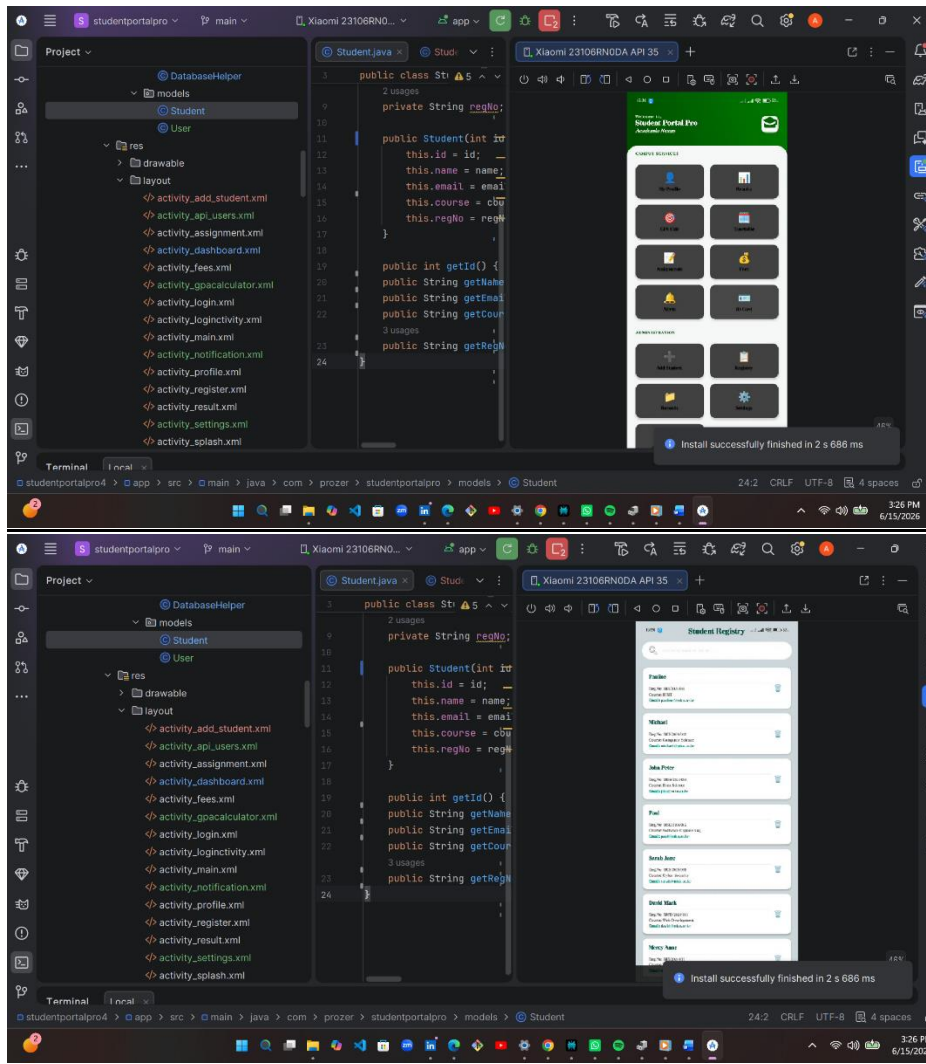
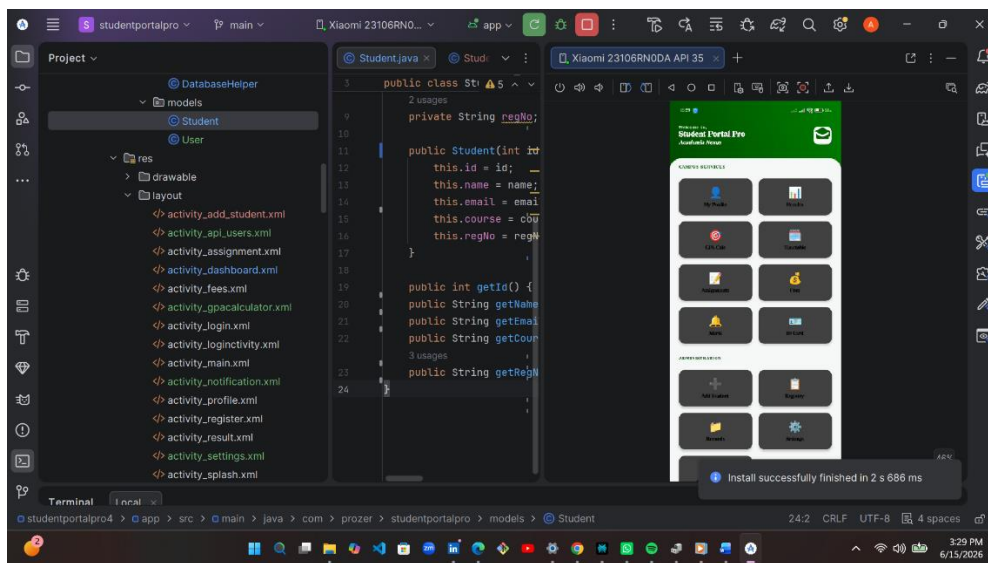


Figure 2. Student registration screen

ADD STUDENT



add student

Figure 3. dashboard with

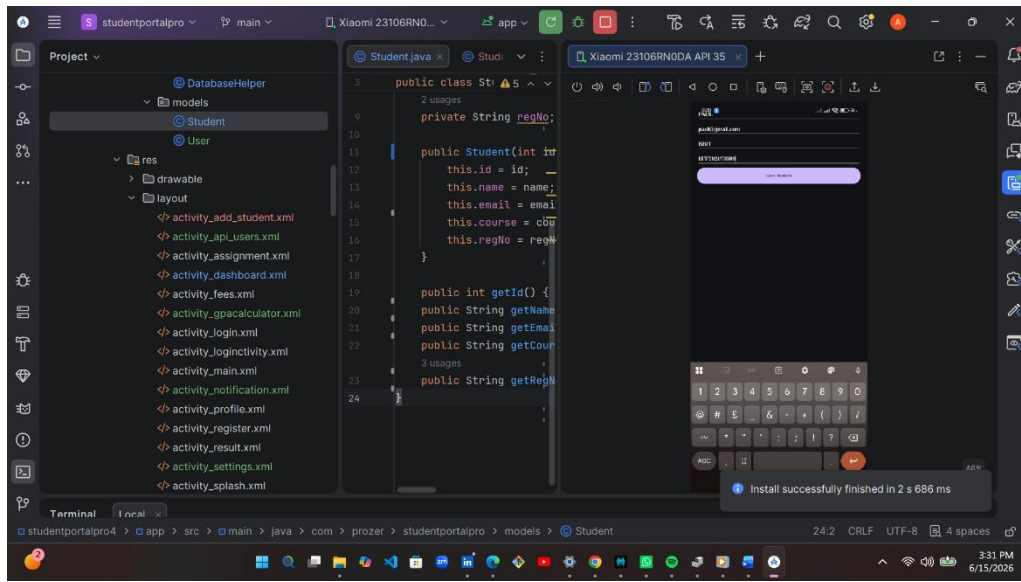


Figure 4.add student

button

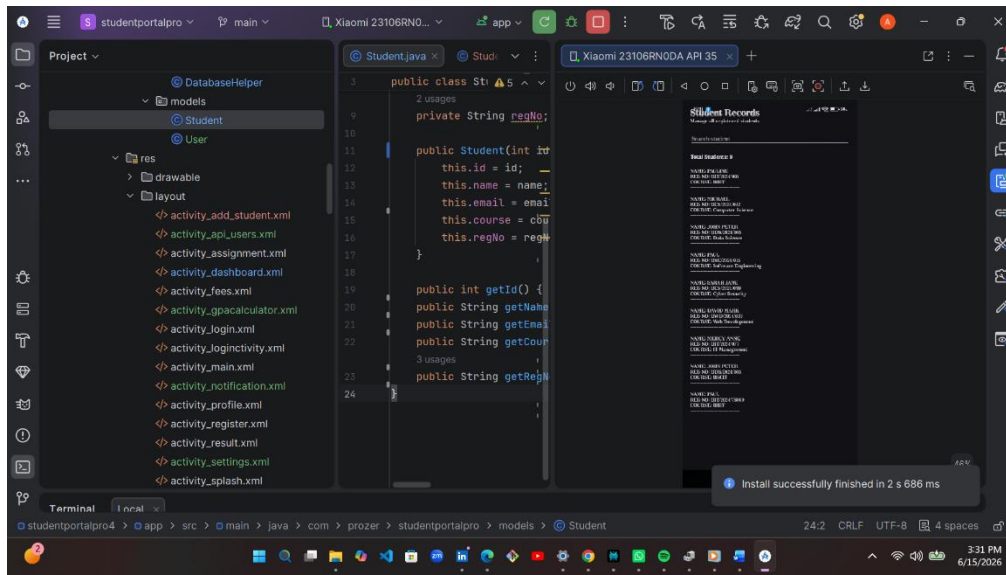


Figure 5.add student on student records

RecyclerView Student List

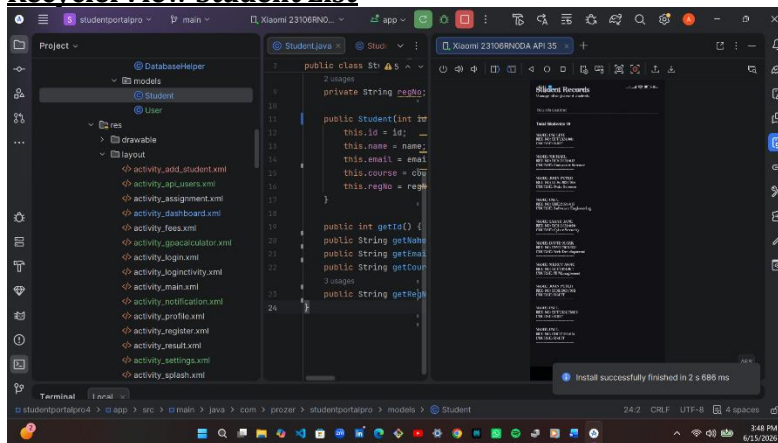


Figure 6.recycle view student list

Search Student

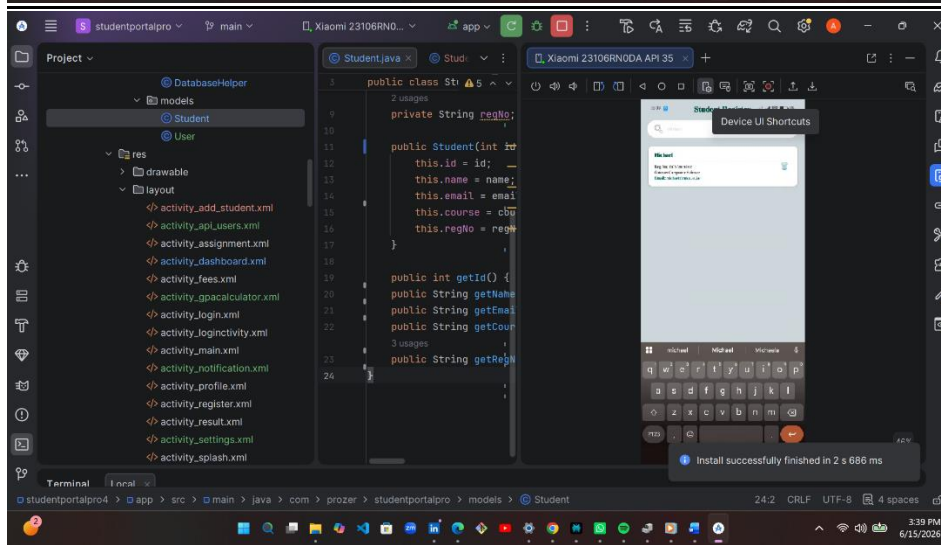
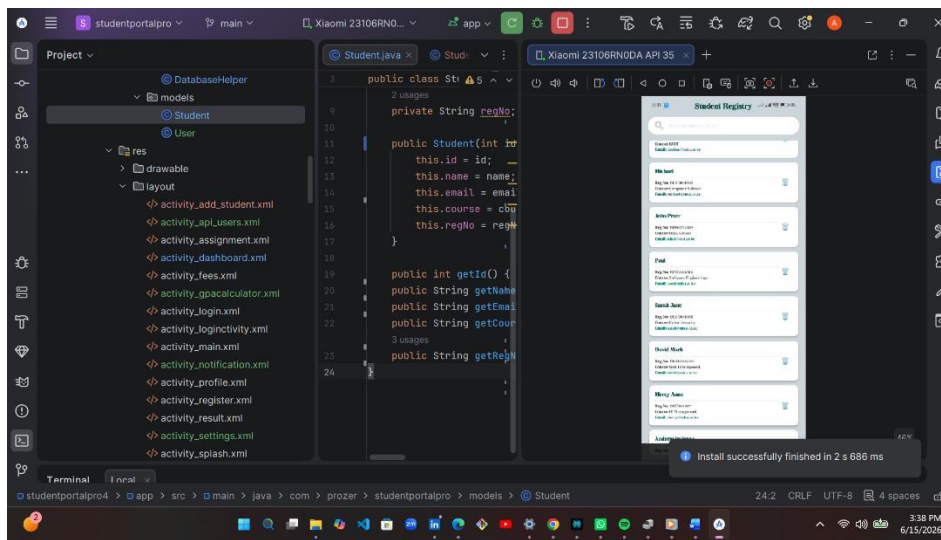
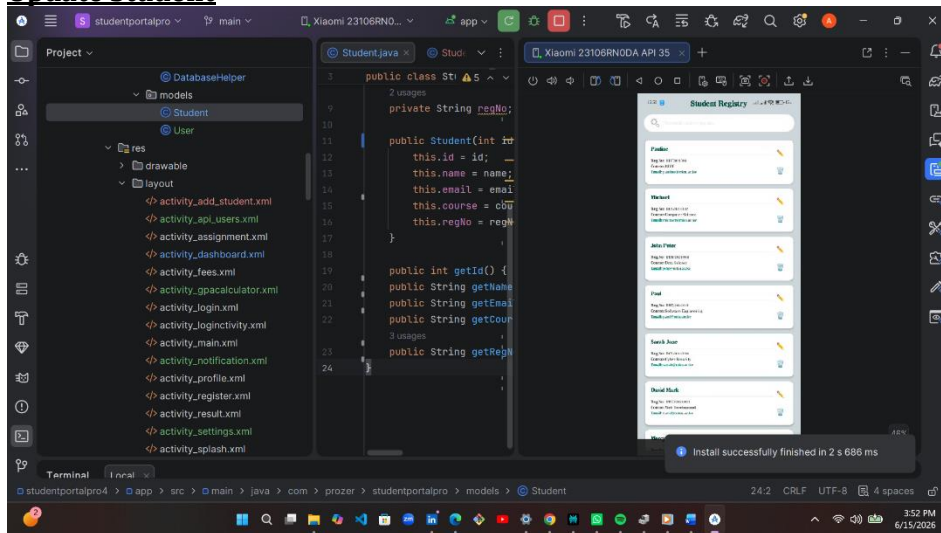


Figure 7.student search functionality

Update Student



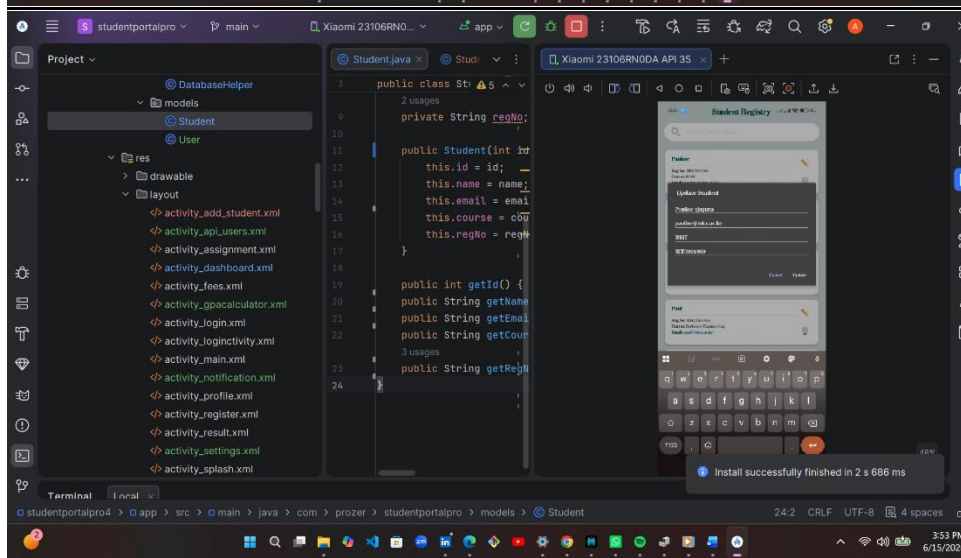
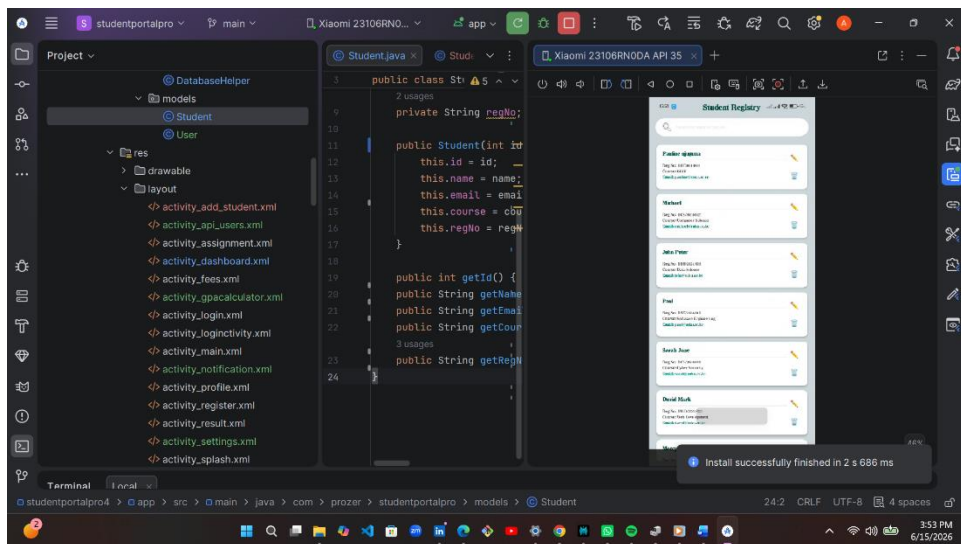
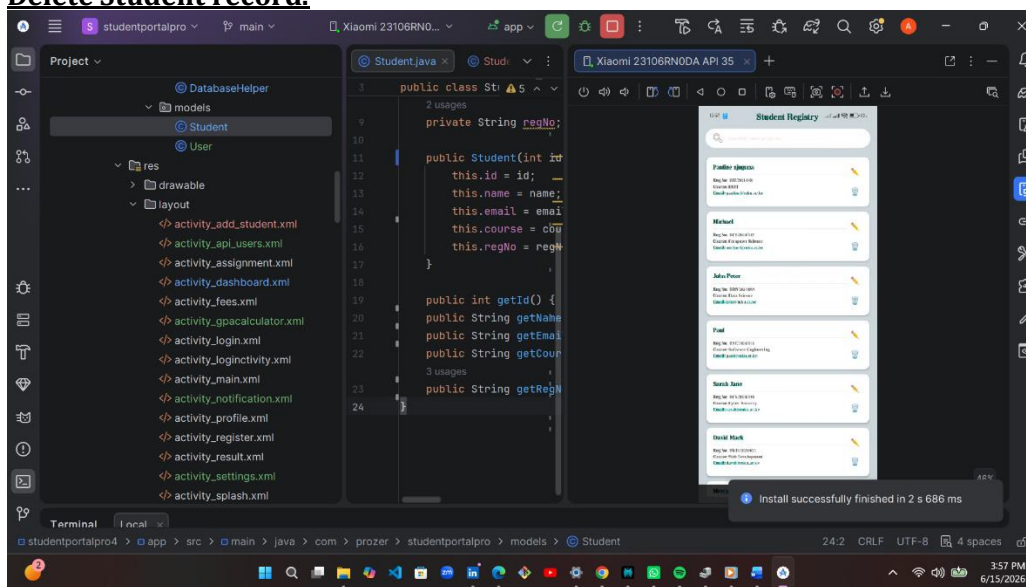


Figure 8 Updating Student Information

Delete Student record.



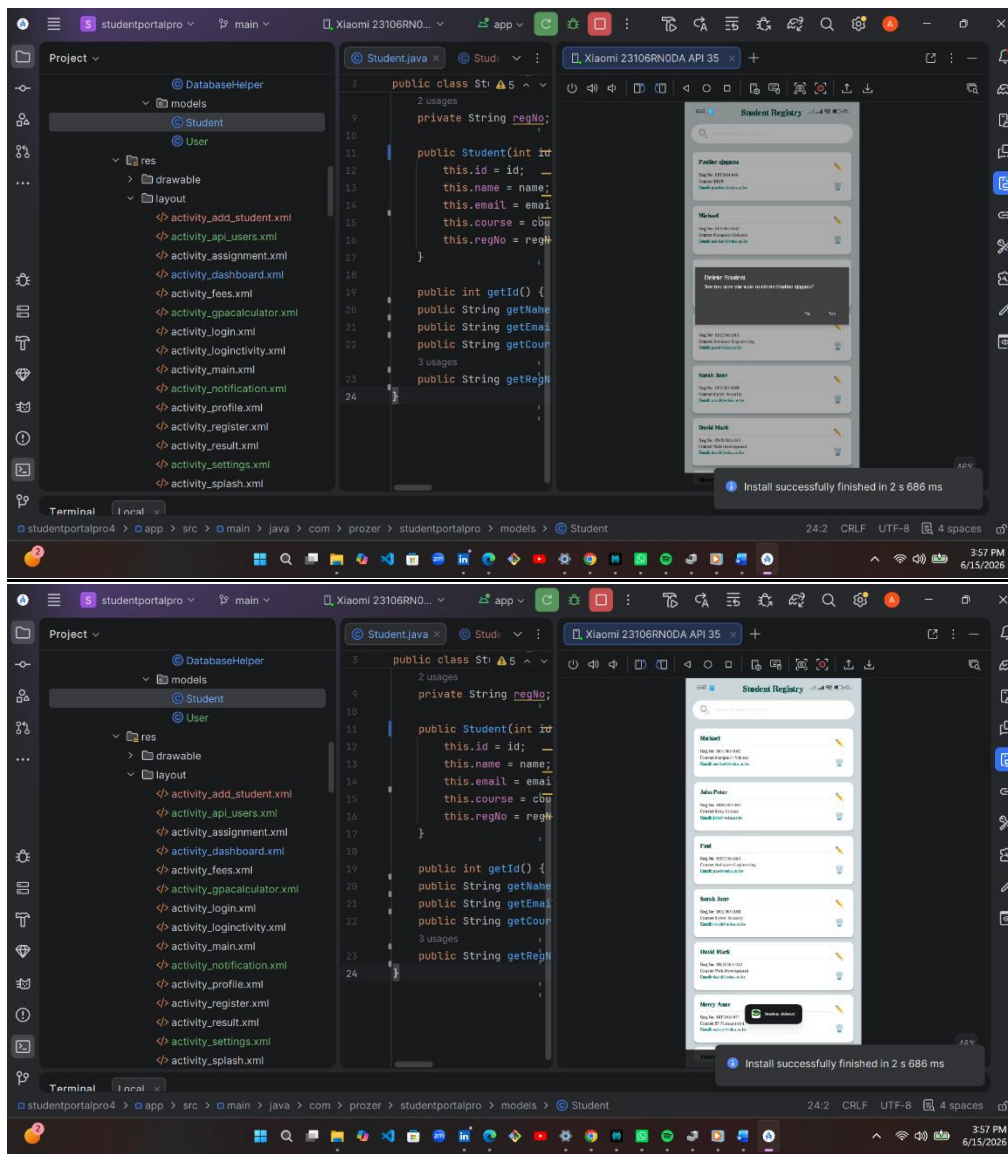


Figure 9 Student Record Deletion

Enhanced Student Portal Dashboard

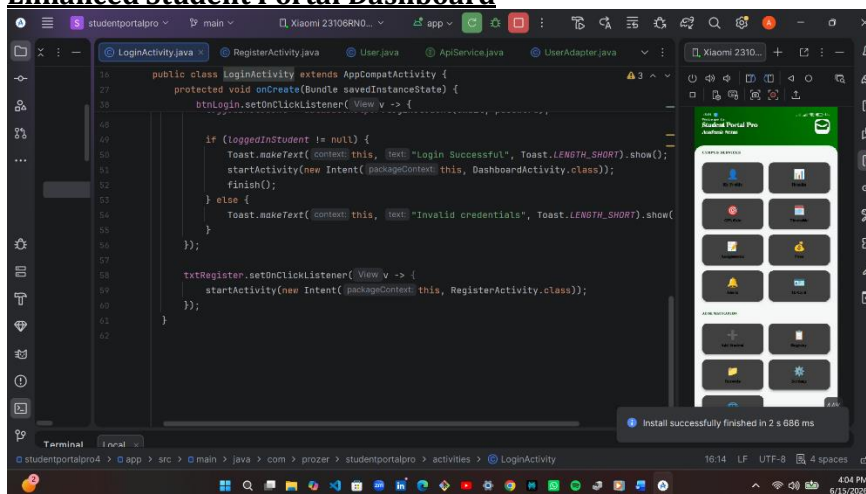


Figure 10 Enhanced Student Portal Dashboard

TOOLS/SOFTWARE USED

1. Android Studio

2. Java
3. SQLite Database
4. Recycler View
5. Shared Preferences
6. Android Emulator

PERSONAL REFLECTION

This week I learned how to manage data using SQLite database in Android applications. I implemented CRUD operations, Recycler View, search functionality, and Shared Preferences, which improved my understanding of data management and user interaction.

Week 5: Data Management

NewsActivity Created

PRACTICAL ACTIVITIES

- Implemented networking functionality using Retrofit.
- Created model classes for handling news data.
- Developed API service interfaces for HTTP requests.
- Configured Retrofit client for server communication.
- Parsed JSON responses and displayed data using RecyclerView.
- Added ProgressBar to indicate loading state.
- Implemented Internet permission and error handling.
- Integrated Campus News feature into the Student Portal Pro application

NewsActivity Created

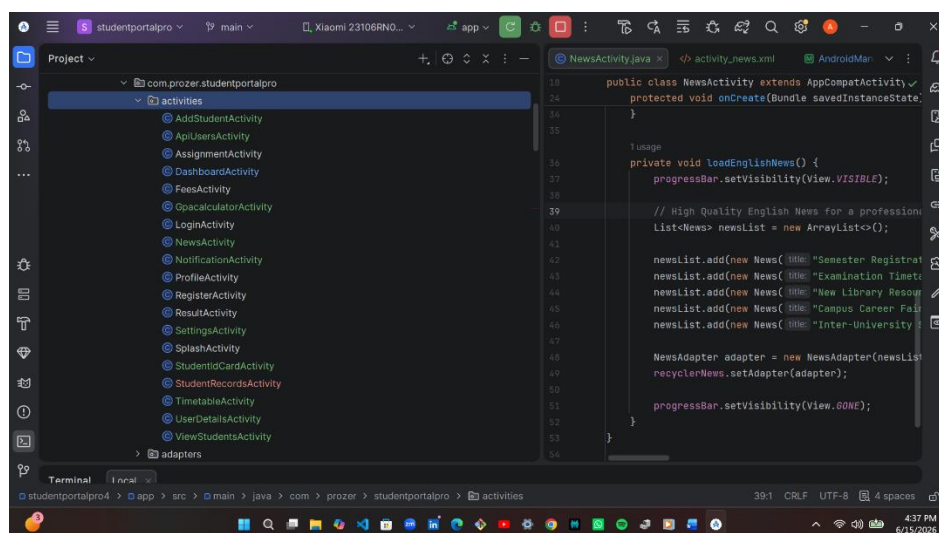


Figure 11, CREATION OF NEWS OF ACTIVITY

News Layout Created

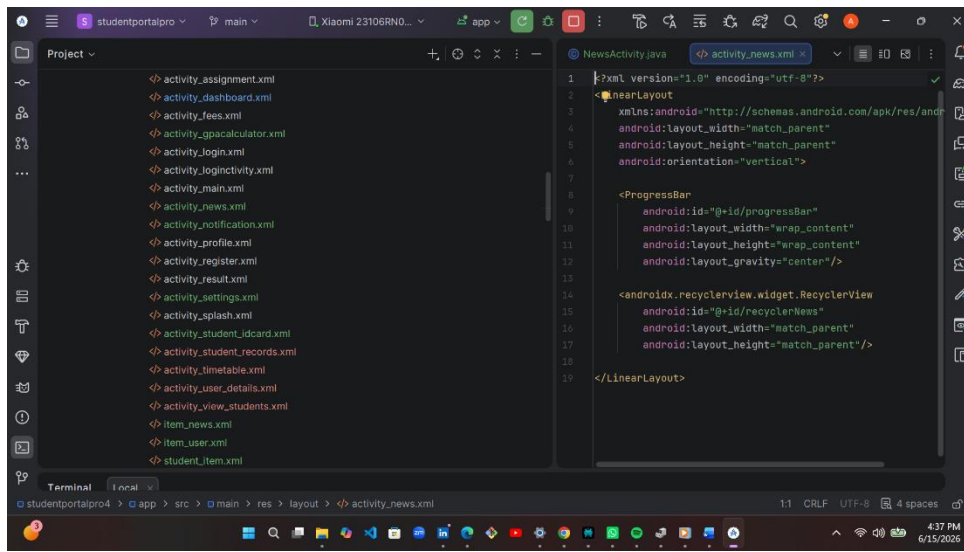


Figure 12 NEWS LAYOUT CREATED

Internet Permission

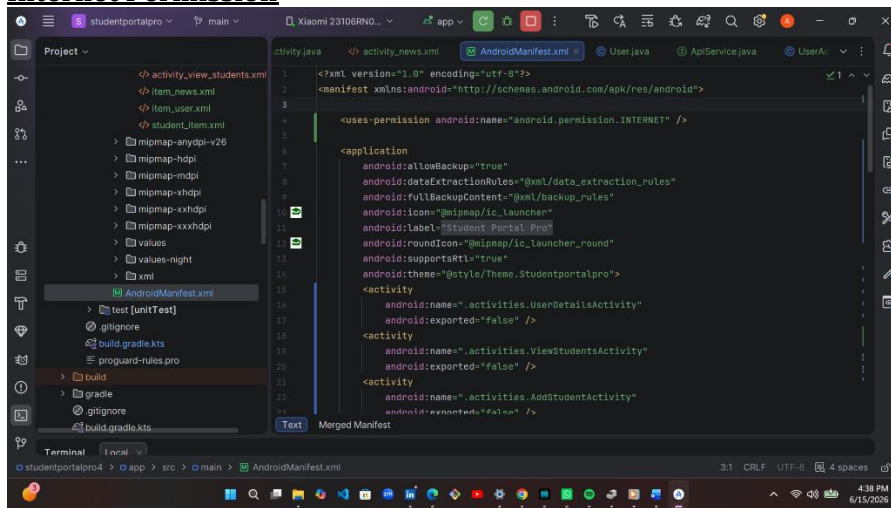


Figure 13.INTERNET PERMISSION

Dashboard Button

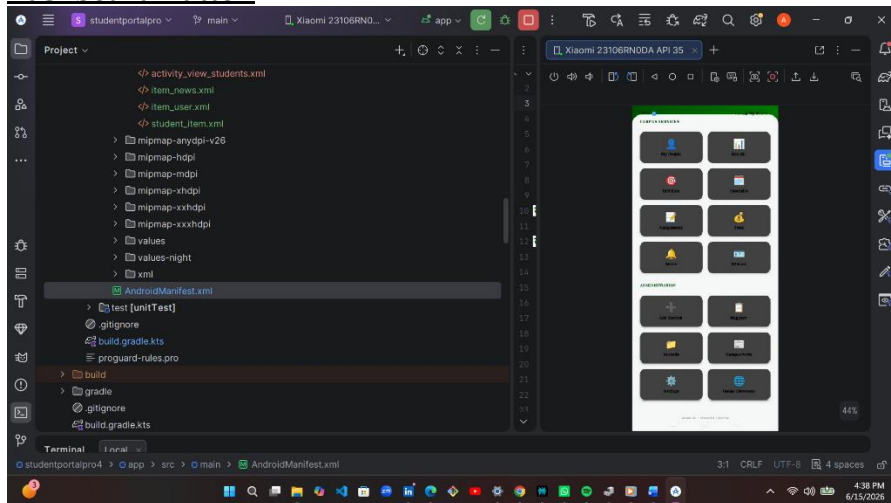


Figure 14.DASHBOARD BUTTON OF CAMPUS NEWS

Campus News Opens

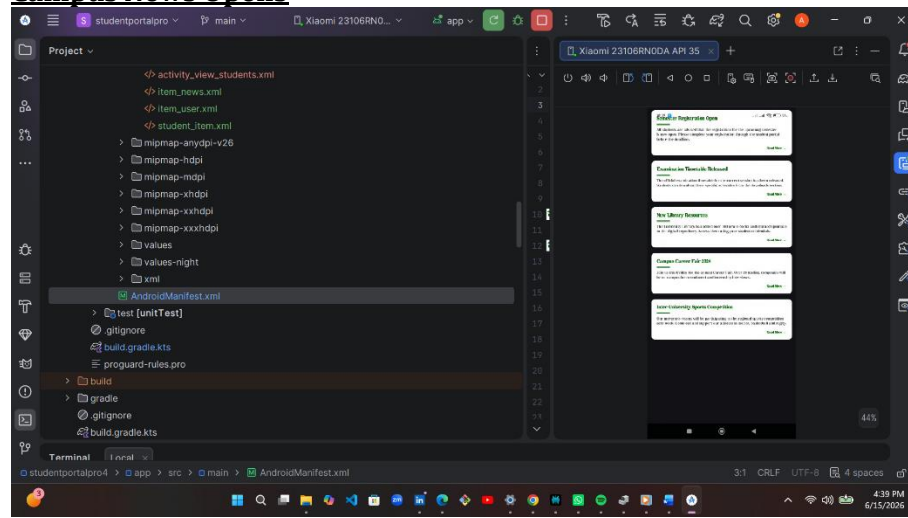
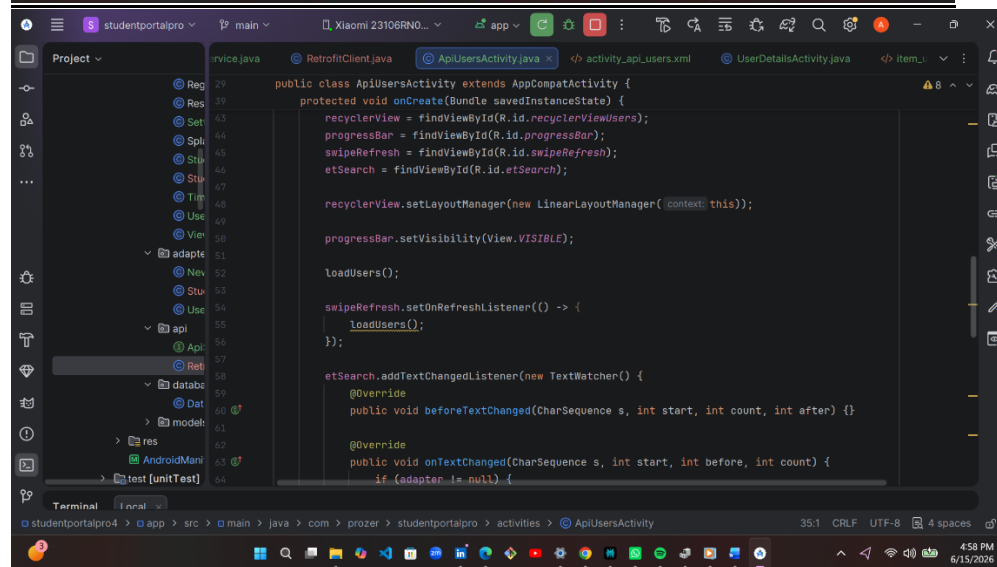
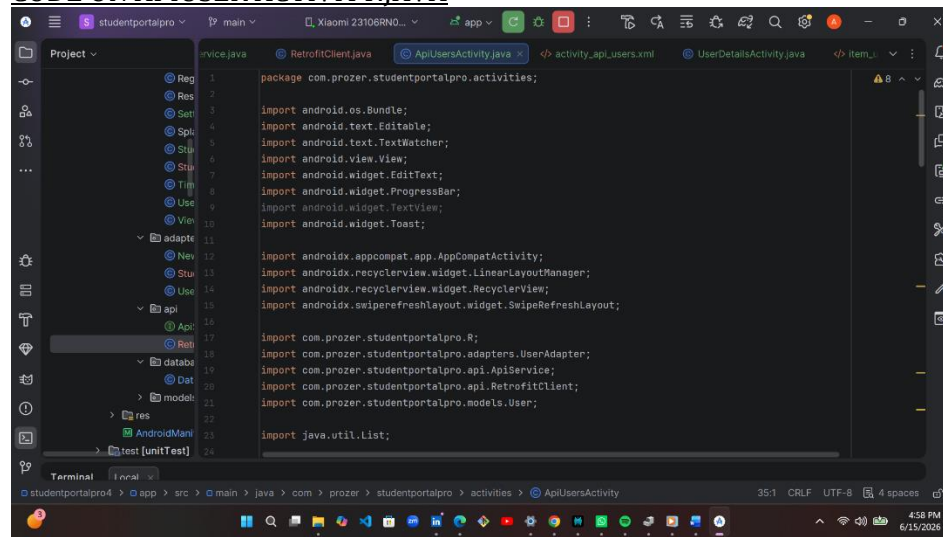


Figure 15NEWS SCREEN INTERFACE

API Service Interface CODE ON APIUSER ACTIVITY.JAVA



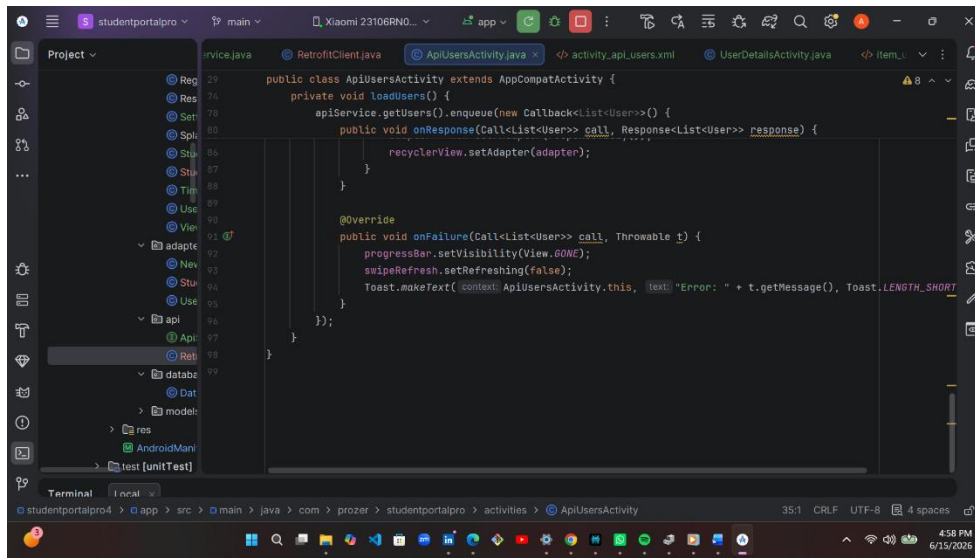
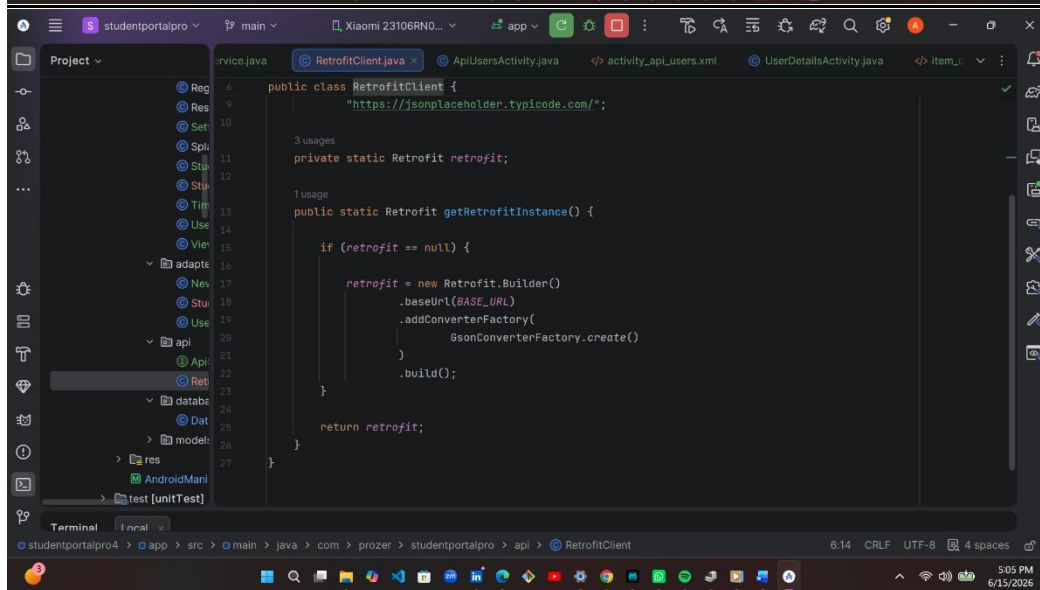
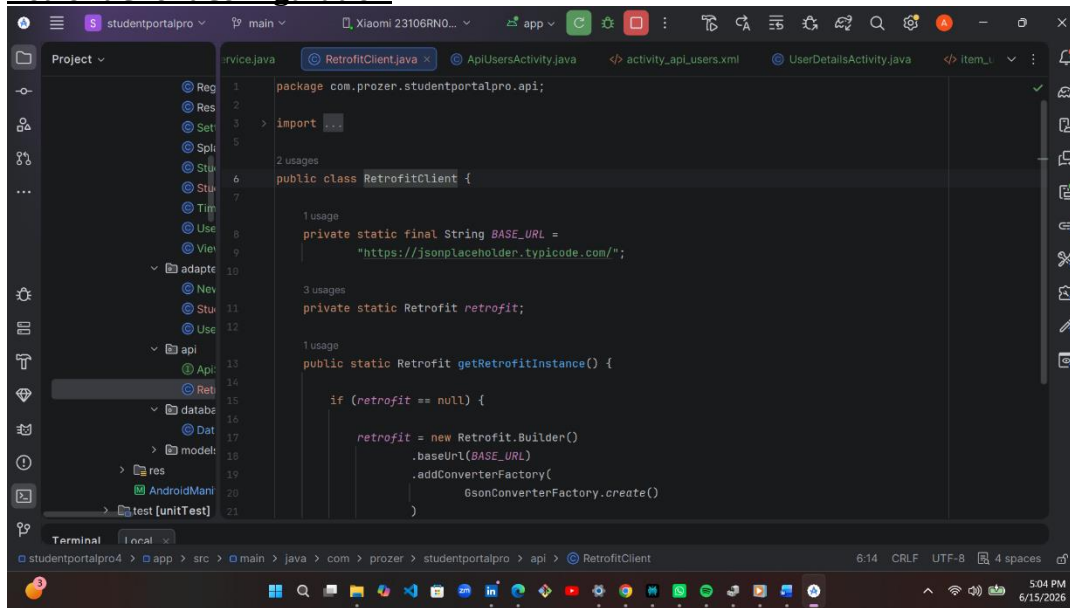


Figure 16 APIUSERACTIVITY.JAVA

Retrofit Client Configuration



*Figure 17*RETROIT CLIENT CONFIGURATION

TOOLS/SOFTWARE USED

Android Studio

Java

Retrofit

RecyclerView

JSON

REST API

Android Emulator

Android SDK

PERSONAL REFLECTION

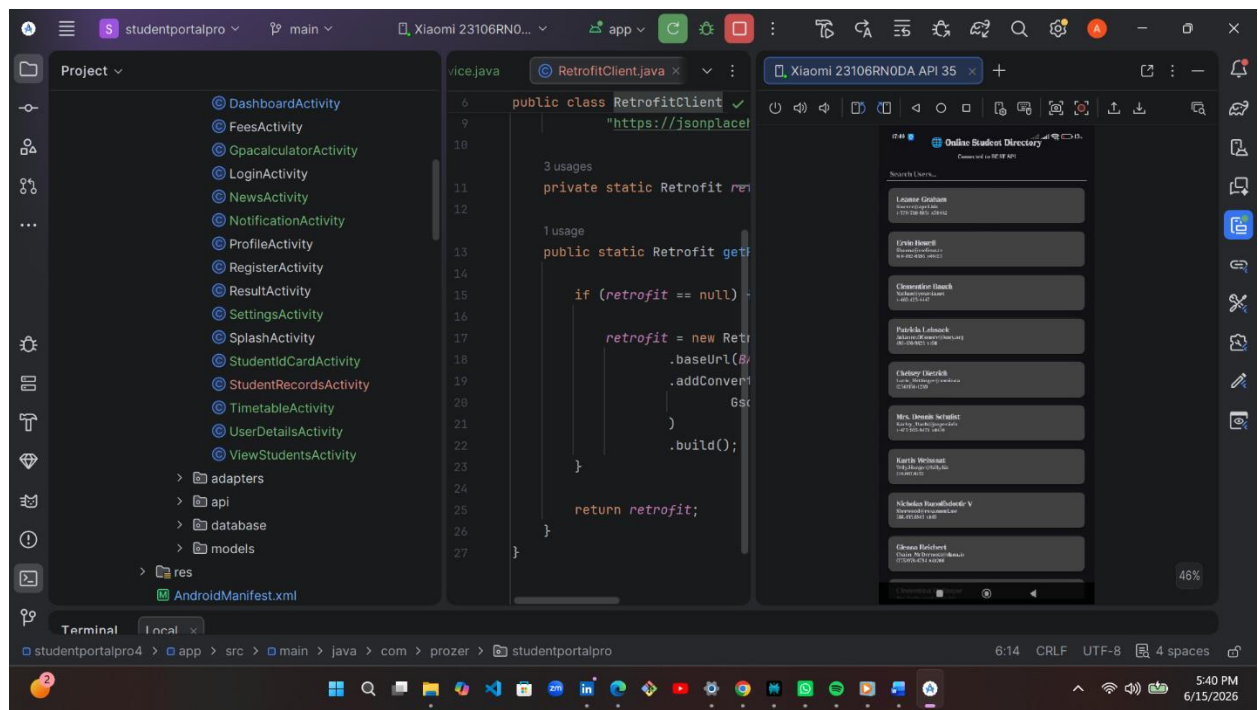
This week I learned how Android applications communicate with remote servers through APIs. I implemented Retrofit networking, JSON parsing, RecyclerView data presentation, and loading indicators. These activities improved my understanding of modern mobile application development and real-time data retrieval.

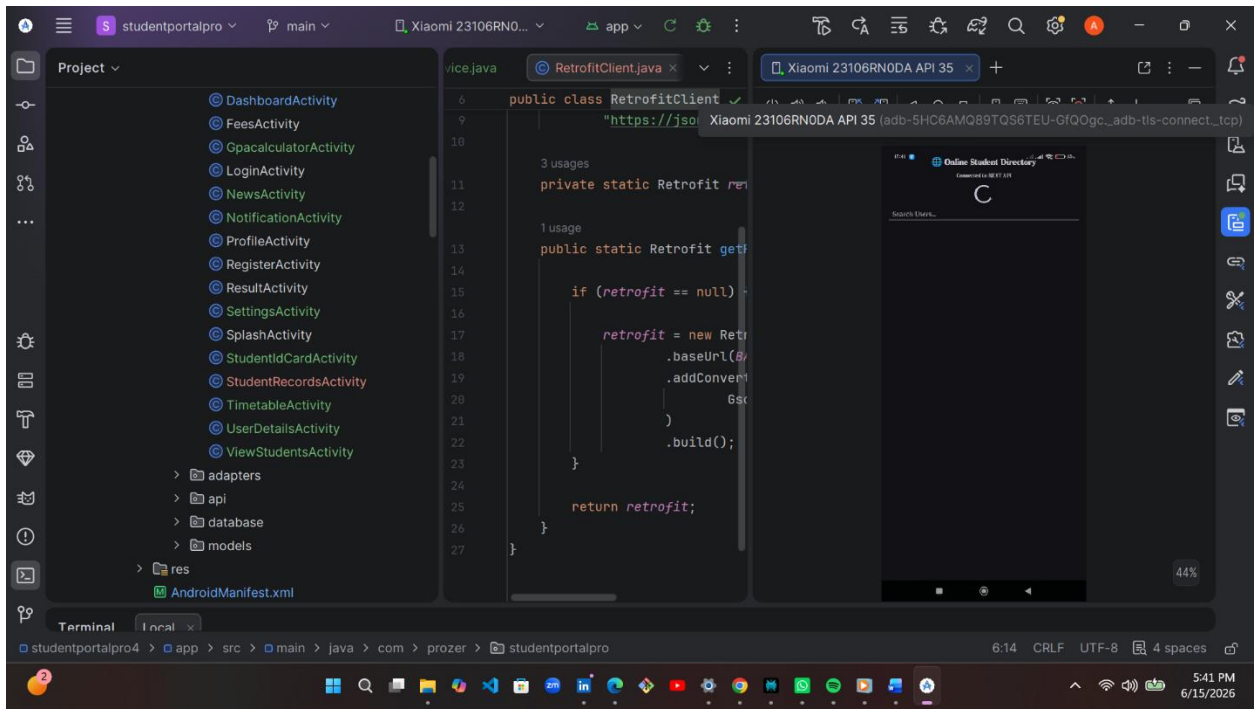
Week 6: Networking and APIs

PRACTICAL ACTIVITIES UNDERTAKEN

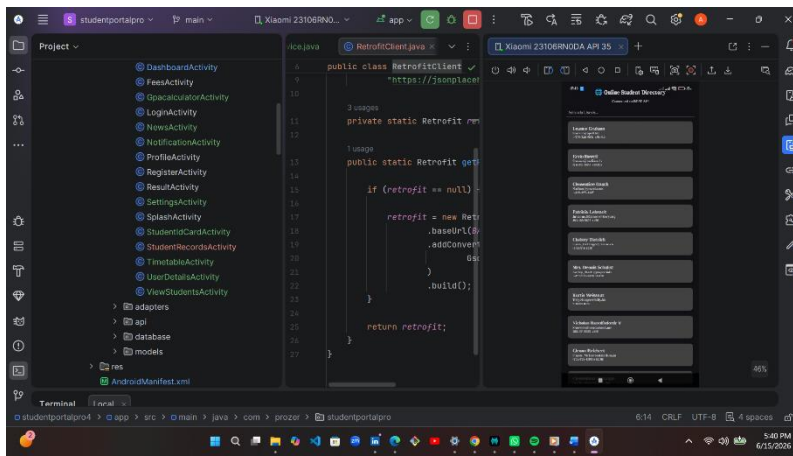
- Implemented an Online Directory module using API communication.
- Retrieved user information from a remote server.
- Displayed fetched data using Recycler View.
- Created model and adapter classes for user management.
- Tested successful network communication and data display.
- Integrated the Online Directory into the Student Portal Pro dashboard.

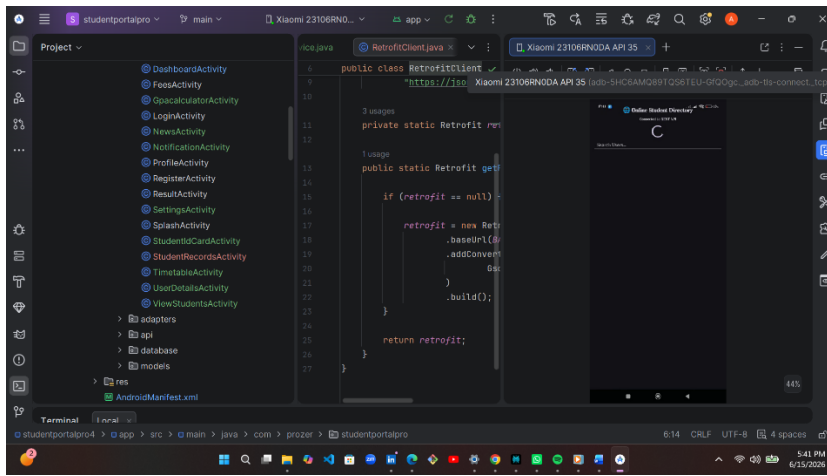
Online Directory Activity Code



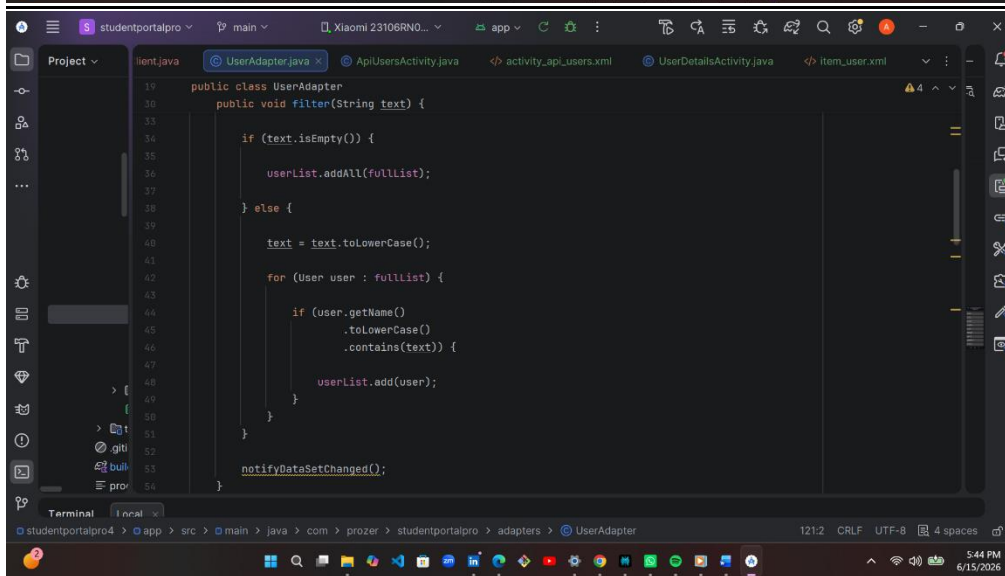
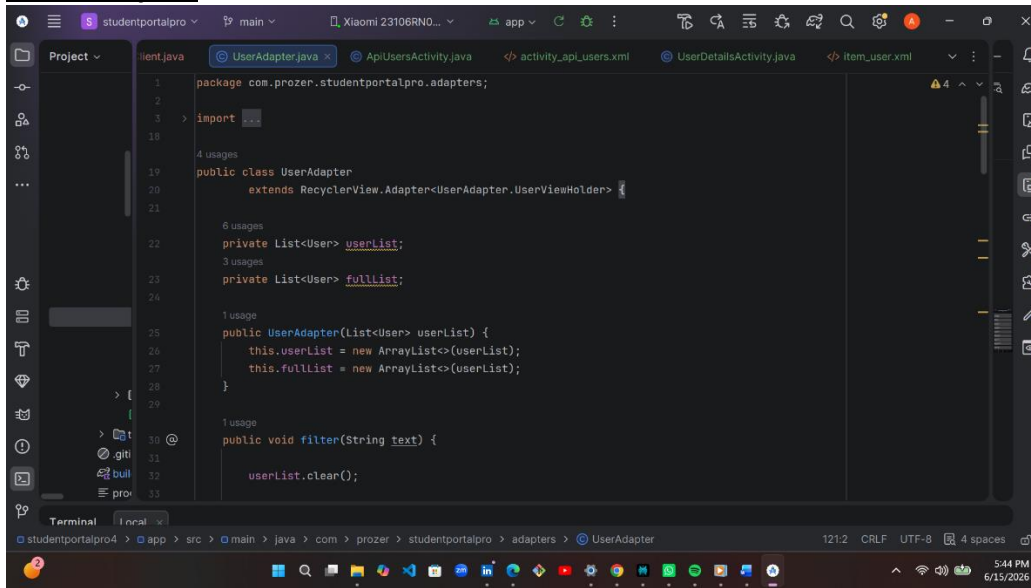


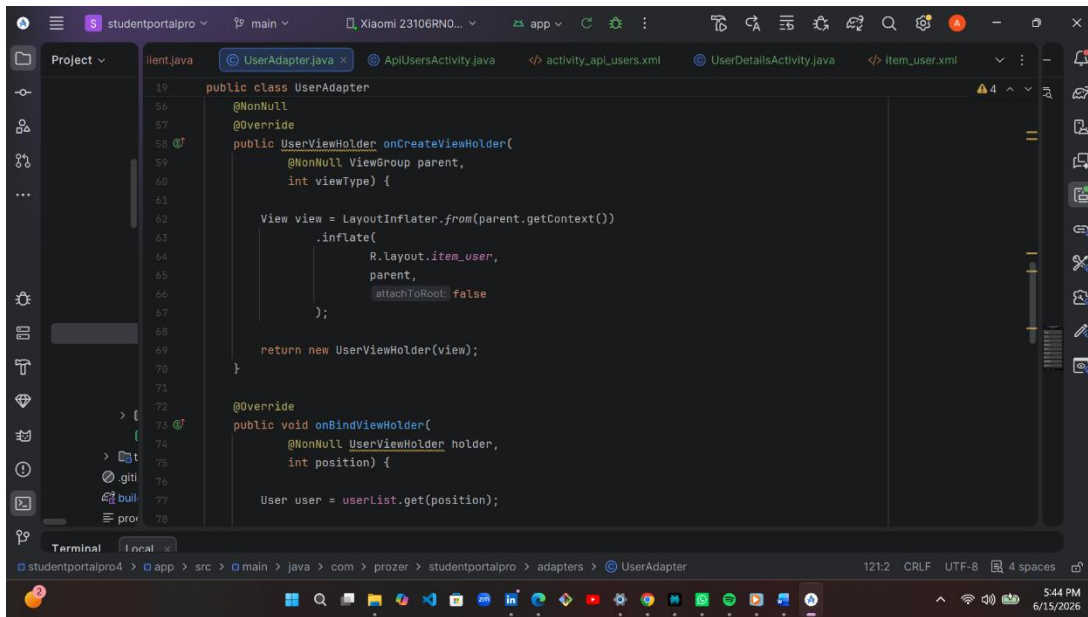
API Call INSIDE ONLINE DIRECTORY





User Adapter



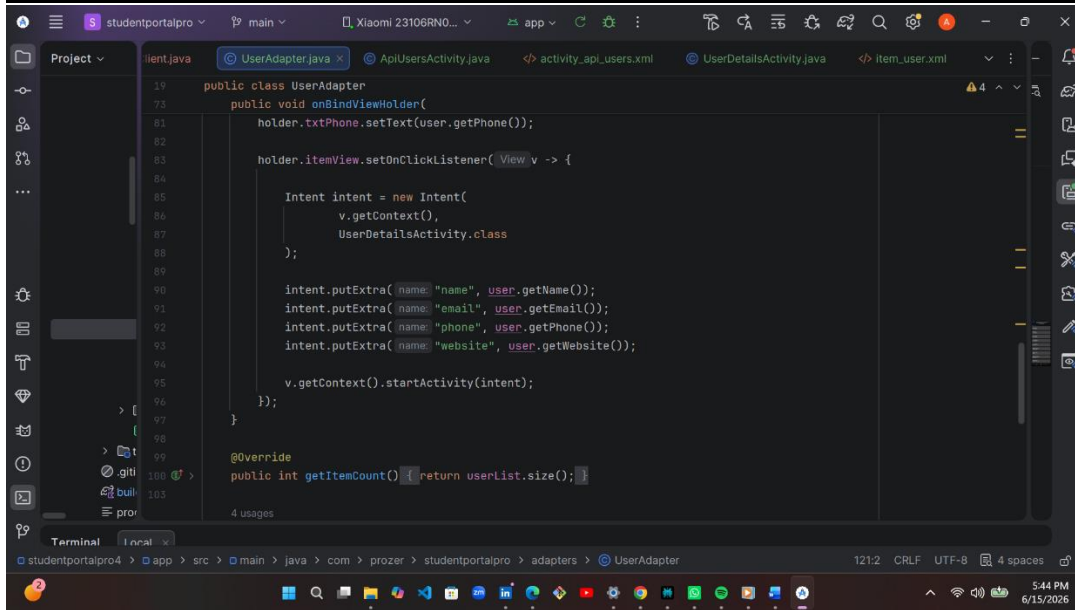


```
19 public class UserAdapter
56     @NonNull
57     @Override
58     public UserViewHolder onCreateViewHolder(
59         @NonNull ViewGroup parent,
60         int viewType) {
61
62         View view = LayoutInflater.from(parent.getContext())
63             .inflate(
64                 R.layout.item_user,
65                 parent,
66                 attachToRoot: false
67             );
68
69         return new UserViewHolder(view);
70     }
71
72     @Override
73     public void onBindViewHolder(
74         @NonNull UserViewHolder holder,
75         int position) {
76
77         User user = userList.get(position);
78     }
```

Terminal Local

studentportalpro4 > app > src > main > java > com > prozer > studentportalpro > adapters > UserAdapter 121:2 CRLF UTF-8 4 spaces

5:44 PM 6/15/2026

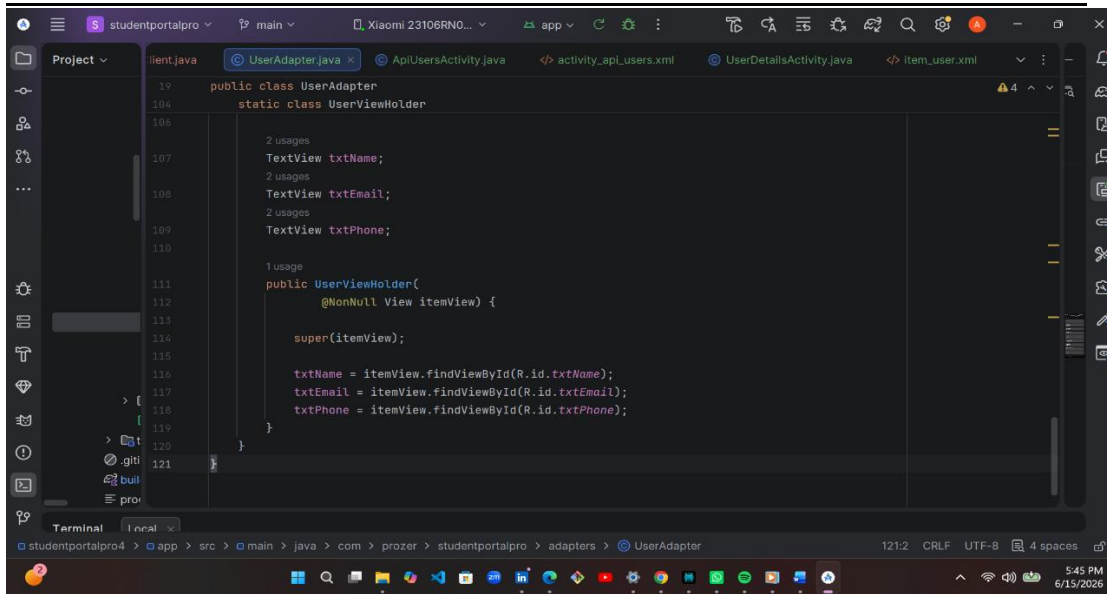


```
73     public void onBindViewHolder(
81         holder.txtPhone.setText(user.getPhone());
82
83         holder.itemView.setOnClickListener( View v -> {
84
85             Intent intent = new Intent(
86                 v.getContext(),
87                 UserDetailsActivity.class
88             );
89
90             intent.putExtra( name: "name", user.getName());
91             intent.putExtra( name: "email", user.getEmail());
92             intent.putExtra( name: "phone", user.getPhone());
93             intent.putExtra( name: "website", user.getWebsite());
94
95             v.getContext().startActivity(intent);
96         });
97     }
98
99     @Override
100     public int getItemCount() { return userList.size(); }
101 }
```

Terminal Local

studentportalpro4 > app > src > main > java > com > prozer > studentportalpro > adapters > UserAdapter 121:2 CRLF UTF-8 4 spaces

5:44 PM 6/15/2026



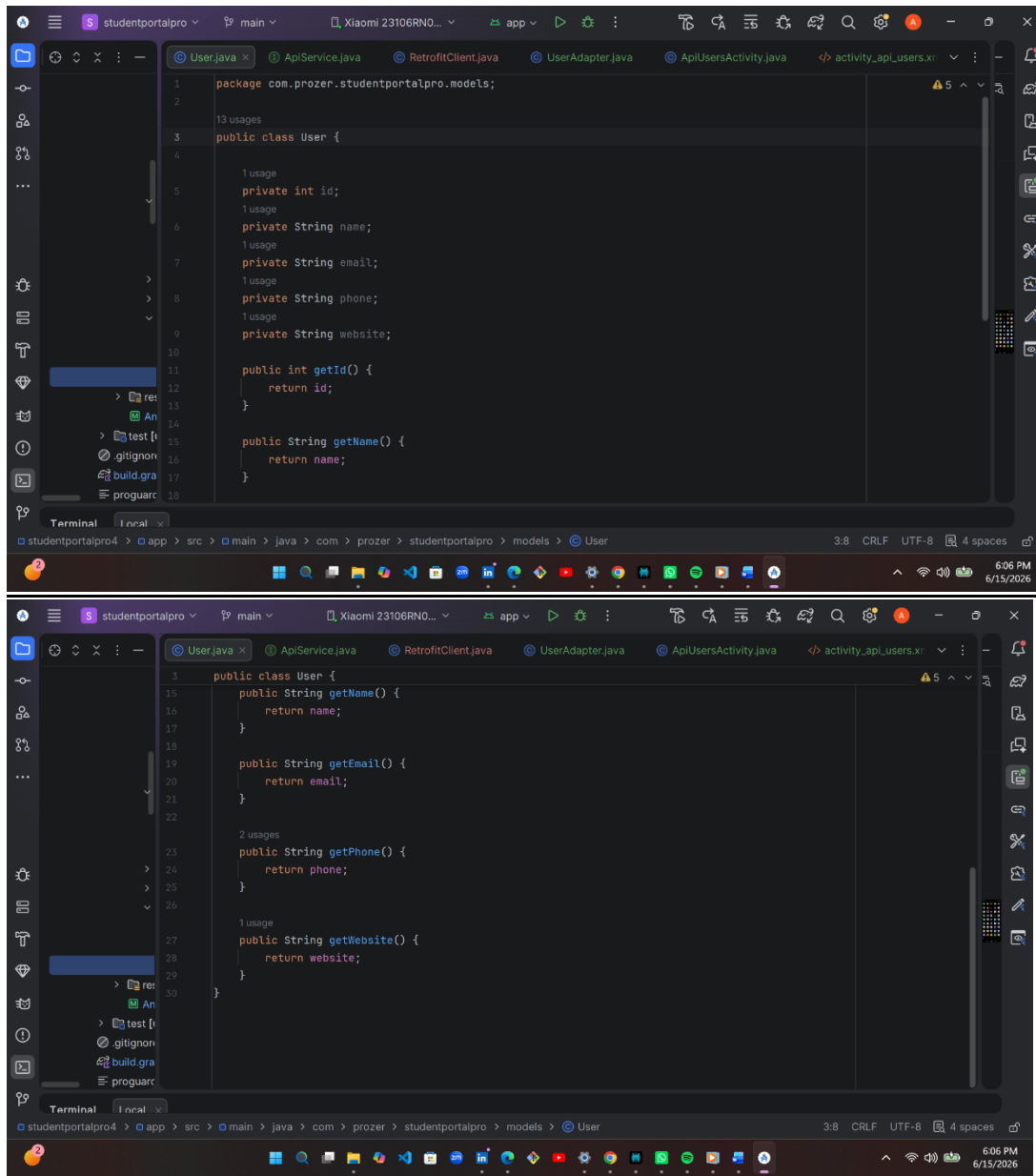
```
19 public class UserAdapter
104     static class UserViewHolder
105     {
106         2 usages
107         TextView txtName;
108         2 usages
109         TextView txtEmail;
110         2 usages
111         TextView txtPhone;
112
113         1 usage
114         public UserViewHolder(
115             @NonNull View itemView) {
116             super(itemView);
117
118             txtName = itemView.findViewById(R.id.txtName);
119             txtEmail = itemView.findViewById(R.id.txtEmail);
120             txtPhone = itemView.findViewById(R.id.txtPhone);
121         }
122     }
```

Terminal Local

studentportalpro4 > app > src > main > java > com > prozer > studentportalpro > adapters > UserAdapter 121:2 CRLF UTF-8 4 spaces

5:45 PM 6/15/2026

User Model



TOOLS USED

- Android Studio
- Java
- Retrofit

- RecyclerView
- REST API
- JSON
- Android Emulator

PERSONAL REFLECTION

This week I implemented an Online Directory feature that retrieves user information from a remote server through APIs. I learned how network requests are made, how data is processed, and how information can be displayed dynamically in Android applications. This improved my understanding of networking concepts and API integration.

Week 7: Application Architecture

Same as week 1

Week 8: Project Planning

Week 9: Device Features and Sensors

Same as week 1

Week 10: Notifications and Background Services

Same as week 1

Week 11: Mobile Security

Same as week 1

Week 12: Testing and Debugging

Same as week 1

Week 13: Application Deployment

Same as week 1

Week 14: Final Project Presentation

Same as week 1

Semester Project Tracking

Project Title

Project Objectives

Project Requirements

Development Milestones

Testing Results

Lessons Learned

Future Improvements

End of Semester Reflection

Summary of Skills Acquired

Most Interesting Topic

Major Challenges Faced

How Challenges Were Overcome

Professional Growth Achieved

Overall Course Experience

Final Approval

Student Name: _____

Student Signature: _____

Date: _____

Lecturer Name: _____

Lecturer Remarks: _____

Date: _____